UNIVERSITY OF PISA

COMPUTER SCIENCE - ARTIFICIAL INTELLIGENCE



Project Track 19 — Non-ML

Computational Mathematics for Learning and Data Analysis Project

QR Factorization and Limited-Memory Quasi-Newton Methods for Regularized Least Squares

Student:
Karol Jinet Cardona Bolanos
Mat. 702023

Student:
Gianluca Panzani
Mat. 550358

ACADEMIC YEAR 2025/2026

Contents

1 Introduction **3**

2 Problem Properties **4**

2.1 Gradient and Hessian 4

2.2 Strong Convexity and Smoothness 4

2.3 Condition Number 5

3 Limited-Memory BFGS (L-BFGS) **6**

3.1 Quasi-Newton Methods: Background 6

3.2 The L-BFGS Algorithm 7

3.2.1 Initial Scaling of H_k^0 7

3.2.2 Two-Loop Recursion 8

3.2.3 Curvature Safeguard and Optional Restart 9

3.3 Line Search Strategies 10

3.3.1 Strong Wolfe Conditions 10

3.3.2 Exact Line Search for Quadratic Objectives 10

3.4 Complete Algorithm 11

3.5 Stopping Criterion 11

3.6 Convergence Analysis 12

3.6.1 Global Linear Convergence 12

3.6.2 Superlinear Convergence 12

3.6.3 Finite Termination on the Quadratic 12

4 L-BFGS Experimental Results **14**

4.1 Theoretical Convergence Properties 14

4.1.1 Empirical Convergence Rates and Finite Termination 14

4.1.2 Comparison with Steepest Descent and Conjugate Gradient 15

4.1.3 Alignment with the Newton Direction 17

4.2 Algorithmic Design Choices 17

4.2.1 Exact Line Search vs. Strong Wolfe Line Search 17

4.2.2 Sensitivity to the Wolfe Constants c_1, c_2 18

4.2.3 Effect of the Memory Parameter $\bar{m}$ 18

4.2.4 Effect of the Regularization Parameter λ 20

4.2.5 H_0 Scaling Strategies 20

4.2.6 Curvature-Based Restart 21

4.3 Numerical and Computational Scalability 22

4.3.1 Sensitivity to the Initial Iterate w_0 22

4.3.2 Theoretical and Actual Cost, Storage . . . 22
4.3.3 Empirical Scaling with the Dimensions n and m . . . 23
4.4 Summary . . . 25

5 QR Factorization **27**
5.1 QR Solution of the Regularized Least-Squares Problem . . . 27
5.1.1 Thin QR Factorization . . . 27
5.1.2 Algorithmic Structure . . . 28
5.1.3 Cost for the QR Solver . . . 29
5.2 QR with Householder Reflectors . . . 29
5.2.1 Dense Householder QR . . . 30
5.2.2 Structure-Based Householder QR . . . 31

6 QR Factorization Experiments **34**
6.1 Runtime Scalability . . . 34
6.1.1 Detailed Operation Count . . . 34
6.1.2 Asymptotic Cost and Timings . . . 36
6.2 Algorithmic Stability and the Condition Number . . . 37
6.3 Summary . . . 39

7 Comparison: QR vs. L-BFGS **41**
7.1 Experimental Protocol and Accuracy Measures . . . 41
7.2 Accuracy and Runtime on ML-CUP25 . . . 42
7.2.1 Time at a Common Accuracy Requirement . . . 42
7.3 Dimensional Scaling and Storage . . . 44
7.3.1 Increasing the Number of Unknowns . . . 44
7.3.2 Increasing the Number of Spectral Directions . . . 45
7.4 Conditioning and a Difficult Synthetic Spectrum . . . 45
7.5 Conclusions . . . 46

A Proofs **47**

Bibliography **48**

Chapter 1

Introduction

This project addresses the numerical solution of a regularized linear least squares problem. Given a data matrix $X \in \mathbb{R}^{m \times n}$ from the ML-CUP dataset, a random response vector $y \in \mathbb{R}^n$, and a regularization parameter $\lambda > 0$, the goal is to find the vector $w \in \mathbb{R}^m$ that minimizes

$$\min_w \left\| \hat{X} w - \hat{y} \right\|, \tag{1.1}$$

where the augmented matrix and vector are defined as

$$\hat{X} = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} \in \mathbb{R}^{(n+m) \times m}, \qquad \hat{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \in \mathbb{R}^{n+m}. \tag{1.2}$$

Expanding the squared norm, the problem is equivalent to minimizing

$$f(w) = \frac{1}{2} \left\| X^T w - y \right\|^2 + \frac{1}{2} \lambda^2 \left\| w \right\|^2, \tag{1.3}$$

which is a standard instance of Tikhonov regularization (ridge regression). The regularization term $\lambda^2 \|w\|^2$ penalizes large coefficient vectors, improving the numerical stability of the solution and controlling overfitting.

Two computational strategies are explored:

- **(A1)** A limited-memory quasi-Newton method (L-BFGS), which iteratively approximates the curvature of the objective function using only gradient information and a small number of stored vector pairs.
- **(A2)** A direct method based on thin QR factorization with Householder reflectors, in the compact variant where the orthogonal matrix Q is not formed explicitly: the stored Householder vectors are used to compute products with Q and Q^T implicitly.

Both algorithms are implemented from scratch, without relying on any off-the-shelf numerical solvers; library routines are used only to compute reference solutions for the experiments.

Chapter 2

Problem Properties

Before discussing the optimization algorithms, we analyze the mathematical properties of the objective function (1.3) that are relevant for convergence.

2.1 Gradient and Hessian

The objective (1.3) is a quadratic function of w. Consequently, its Hessian matrix is constant, as it does not depend on the variable w but only on the fixed data matrix X and the regularization parameter λ. Its gradient and Hessian are:

$$\nabla f(w) = X(X^T w - y) + \lambda^2 w, \tag{2.1}$$

$$H \coloneqq \nabla^2 f(w) = XX^T + \lambda^2 I_m. \tag{2.2}$$

2.2 Strong Convexity and Smoothness

To guarantee the existence of a unique global minimum and to analyze the convergence speed of our optimization algorithms, we first establish the strong convexity and smoothness of the objective function.

By Lemma 1, the Hessian $H = XX^T + \lambda^2 I_m$ is symmetric positive definite whenever $\lambda > 0$. This confirms that $f(w)$ is strictly convex, ensuring that any local minimum is also the unique global minimum.

Formally, this implies that f is μ-strongly convex and L-smooth, where the constants μ and L correspond respectively to the smallest and largest eigenvalues of H. By Lemma 2 (eigenvalue shift), the eigenvalues of $H = XX^T + \lambda^2 I_m$ are obtained by shifting those of XX^T by λ^2:

$$\mu = \lambda_{\min}(H) = \lambda_{\min}(XX^T) + \lambda^2, \qquad L = \lambda_{\max}(H) = \lambda_{\max}(XX^T) + \lambda^2. \tag{2.3}$$

The smoothness constant L bounds how rapidly the gradient can change. For all $w, w' \in \mathbb{R}^m$:

$$\left\|\nabla f(w) - \nabla f(w')\right\| = \left\|H(w - w')\right\| \leq \|H\| \left\|w - w'\right\| = L \left\|w - w'\right\|. \tag{2.4}$$

Since our data matrix $X \in \mathbb{R}^{m \times n}$ is tall-thin ($m > n$), by Lemma 3 the eigenvalues of XX^T are $\{\sigma_1^2, \ldots, \sigma_n^2, 0, \ldots, 0\}$, so in particular $\lambda_{\min}(XX^T) = 0$. Therefore, the strong convexity constant simplifies to:

$$\mu = \lambda^2. \tag{2.5}$$

This highlights the crucial role of λ: without regularization, H would be only positive *semi*-definite, $\mu = 0$, and f would lack strong convexity. In that case, the system $Hw = Xy$ would be singular with infinitely many solutions, the minimum would not be unique, and the limit reached by an iterative method would depend on its starting point.

Together, L and μ quantify the curvature of the objective: μ controls the minimum curvature (how sharply f rises in the flattest direction) and L the maximum curvature (how sharply it rises in the steepest direction). Their ratio L/μ is the condition number of the Hessian, $\kappa(H) = \kappa(\hat{X})^2$, and its square root is the condition number $\kappa(\hat{X})$ of the augmented matrix derived in Section 2.3. It governs the worst-case convergence speed of iterative solvers; for L-BFGS on our quadratic a stronger result holds (Section 3.6.3).

2.3 Condition Number

The geometric shape of the objective function—and consequently the difficulty of the optimization problem—is summarized by the condition number of the augmented matrix $\hat{X}$, defined as the ratio of its largest to smallest singular values.

To express this in terms of the problem parameters, we note that the singular values of $\hat{X}$ are the square roots of the eigenvalues of $\hat{X}^T\hat{X}$. A direct computation gives:

$$\hat{X}^T\hat{X} = \begin{bmatrix} X & \lambda I_m \end{bmatrix} \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} = XX^T + \lambda^2 I_m = H, \tag{2.6}$$

so $\sigma_i(\hat{X}) = \sqrt{\lambda_i(H)}$ for each i. Combining this with equations (2.3), the condition number of $\hat{X}$ is:

$$\kappa(\hat{X}) = \frac{\sigma_{\max}(\hat{X})}{\sigma_{\min}(\hat{X})} = \sqrt{\frac{\lambda_{\max}(H)}{\lambda_{\min}(H)}} = \sqrt{\frac{L}{\mu}} = \frac{\sqrt{\lambda_{\max}(XX^T) + \lambda^2}}{\lambda}. \tag{2.7}$$

The last step uses $\mu = \lambda^2$, which holds because XX^T has $m - n$ zero eigenvalues when $m > n$ (Lemma 3). The smallest singular value of $\hat{X}$ is therefore λ itself, and *not* the smallest non-zero singular value of X: the regularization block λI_m is what bounds the spectrum of $\hat{X}$ from below.

As the equation shows, the condition number depends heavily on the regularization parameter. For small values of λ, it scales as $\mathcal{O}(1/\lambda)$: weak regularization produces a highly elongated, canyon-like objective landscape, on which first-order methods such as steepest descent take many small zigzagging steps. Conversely, a large λ drives $\kappa(\hat{X}) \to 1$, making the problem nearly spherical and easy to solve from any direction.

The quantity $\kappa(\hat{X})$ measures the conditioning of the problem on the whole of $\mathbb{R}^m$. An iterative method started at $w_0 = 0$ never leaves the range of X (Section 3.6.3), where the Hessian has the n eigenvalues $\sigma_i^2 + \lambda^2$ and the relevant ratio is $(\sigma_{\max}^2 + \lambda^2)/(\sigma_{\min}^2 + \lambda^2)$, which stays bounded as $\lambda \to 0$. This is why the accuracy of L-BFGS does not follow $\kappa(\hat{X})$ (Section 4.2.4), whereas that of a direct solver does: as $\kappa(\hat{X})$ for QR and as $\kappa(\hat{X})^2$ for the normal equations (Section 6.2).

Chapter 3

Limited-Memory BFGS (L-BFGS)

3.1 Quasi-Newton Methods: Background

Quasi-Newton methods are iterative algorithms for unconstrained optimization that approximate the Newton direction without explicitly computing the true Hessian matrix [1]. Recall that the pure Newton step would be $p_k = -[\nabla^2 f(w_k)]^{-1}\nabla f(w_k)$: it accounts for the local curvature and converges quadratically near the solution, but requires computing and factorizing an $m \times m$ matrix at every iteration, which costs $O(m^3)$ and is infeasible for large m. Quasi-Newton methods replace the true inverse Hessian with an approximation $H_k \approx [\nabla^2 f(w_k)]^{-1}$ that is built cheaply from first-order information only. The search direction is then:

$$p_k = -H_k \nabla f(w_k), \tag{3.1}$$

and the iterate is updated as $w_{k+1} = w_k + \alpha_k p_k$, where $\alpha_k > 0$ is the step length determined by a line search.

The secant equation. After each step, the algorithm observes two key vectors: the parameter displacement $s_k = w_{k+1} - w_k$ and the gradient difference $y_k = \nabla f(w_{k+1}) - \nabla f(w_k)$. These encode curvature information: in one dimension, the second derivative can be approximated as $f''(x) \approx \Delta f'/\Delta x = y_k/s_k$. In $\mathbb{R}^m$, this generalizes to the requirement that the new approximation H_{k+1} maps the gradient change y_k to the step s_k, exactly as the true inverse Hessian would near a quadratic:

$$H_{k+1}\, y_k = s_k. \tag{3.2}$$

This is called the *secant condition*. Geometrically, it forces H_{k+1} to reproduce the correct curvature along the direction just explored. However, in dimension $m > 1$, equation (3.2) has infinitely many solutions: it constrains H_{k+1} only along y_k, leaving all orthogonal directions undetermined.

The BFGS update. The BFGS method resolves the non-uniqueness by solving a constrained optimization problem: among all symmetric positive definite matrices satisfying the secant condition, find the one closest to the previous approximation H_k in a weighted Frobenius norm. This *minimal-change* principle ensures that information accumulated in H_k is preserved as much as possible, and only updated in the directions where new curvature has been observed. The unique solution is:

$$H_{k+1} = V_k^T H_k V_k + \rho_k s_k s_k^T, \tag{3.3}$$

where

$$\rho_k = \frac{1}{y_k^T s_k}, \qquad V_k = I - \rho_k y_k s_k^T. \tag{3.4}$$

The structure of this update is highly deliberate. The scalar ρ_k normalizes the curvature so that $\rho_k(s_k^T y_k) = 1$. The matrix V_k acts as a projector that annihilates y_k: since $V_k y_k = y_k - \rho_k y_k(s_k^T y_k) = y_k - y_k = 0$, the first term in (3.3) contributes nothing along y_k. The second term $\rho_k s_k s_k^T$ then injects precisely the curvature needed to satisfy the secant condition. Indeed, multiplying (3.3) by y_k:

$$H_{k+1}y_k = V_k^T H_k \underbrace{(V_k y_k)}_{=0} + \rho_k s_k \underbrace{(s_k^T y_k)}_{=1/\rho_k} = s_k. \checkmark \tag{3.5}$$

While BFGS achieves superlinear convergence on strongly convex problems, it has a fundamental limitation: after k iterations, H_k is a dense $m \times m$ matrix. Storing it requires $O(m^2)$ memory, and updating it via formula (3.3) costs $O(m^2)$ per iteration. For large m, both become prohibitive.

3.2 The L-BFGS Algorithm

The Limited-Memory BFGS (L-BFGS) method addresses the memory limitation with a key observation: we never need to *store* H_k explicitly. What we actually need at each iteration is the matrix-vector product $H_k \nabla f_k$. If we expand the BFGS recurrence (3.3) backwards over the last $\bar{m}$ steps, starting from an initial approximation $H_k^0 = \gamma_k I$, the product $H_k \nabla f_k$ can be expressed as a sequence of inner products and vector additions involving only the stored pairs $\{s_i, y_i\}_{i=k-\bar{m}}^{k-1}$. This leads to the *two-loop recursion* (Algorithm 1), which computes $H_k \nabla f_k$ implicitly in $O(\bar{m}\, m)$ operations, without ever forming or storing H_k.

In practice, $\bar{m} \in [3, 20]$ pairs are retained: Nocedal and Wright [1, Section 9.1] note that modest values of $\bar{m}$ often give satisfactory results, since curvature information from earlier iterations becomes increasingly irrelevant as the iterates move away from those regions. Once $\bar{m}$ pairs are stored, at each iteration the oldest pair is discarded and the newest pair (s_k, y_k) is added. In our implementation the default is $\bar{m} = 12$, equal to n, the effective rank of XX^T: as discussed in Chapter 4, a memory of this size is enough to capture all the non-trivial curvature directions of H.

3.2.1 Initial Scaling of H_k^0

At each iteration, the initial Hessian approximation is set to $H_k^0 = \gamma_k I$ for some scalar $\gamma_k > 0$. Conceptually, this is the "blank slate" from which the two-loop recursion builds H_k by applying the last $\bar{m}$ BFGS updates. The choice of γ_k matters: if it is too small, the search direction p_k will be short and many small steps will be needed; if it is too large, the algorithm will overshoot. We implement three strategies.

Nocedal scaling (BB2). The choice recommended by Nocedal and Wright [1, Eq. 9.6] is

$$\gamma_k^{\mathrm{N}} = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}}. \tag{3.6}$$

This is the unique minimiser of $\|\gamma\, y_{k-1} - s_{k-1}\|^2$ over $\gamma > 0$, i.e. the best scalar approximation to the inverse Hessian along s_{k-1} in a least-squares sense: γ_k^{N} is the scalar γ such that $\gamma y_{k-1} \approx s_{k-1}$, mimicking the secant condition $H_k y_{k-1} = s_{k-1}$. It ensures that the search direction p_k is well-scaled and that the unit step length $\alpha_k = 1$ is accepted in most Wolfe line-search iterations. We note that this formula is algebraically identical to the *BB2* step size of Barzilai and Borwein [2, Eq. (5)], which was derived independently in the context of gradient descent by solving the same least-squares approximation to the secant equation, with $\Delta x \leftrightarrow s_{k-1}$ and $\Delta g \leftrightarrow y_{k-1}$. This observation motivates us to also consider the companion BB1 step as an alternative scaling strategy for H_k^0.

BB1 scaling. The companion step size proposed by Barzilai and Borwein [2, Eq. (6)] is

$$\gamma_k^{\mathrm{BB1}} = \frac{s_{k-1}^T s_{k-1}}{s_{k-1}^T y_{k-1}}. \tag{3.7}$$

This is the minimiser of $\|\gamma^{-1} s_{k-1} - y_{k-1}\|^2$, i.e. the scalar γ such that $\gamma^{-1} s_{k-1} \approx y_{k-1} \approx H\, s_{k-1}$, so that γ^{-1} estimates an average eigenvalue of $\nabla^2 f$. By the Cauchy–Schwarz inequality, $\gamma_k^{\mathrm{BB1}} \geq \gamma_k^{\mathrm{N}}$ always holds, so BB1 produces larger steps than BB2, which can accelerate convergence by escaping flat regions more quickly. BB1 is the default scaling of our implementation.

Safeguarded BB2. Near the solution both $s_{k-1}^T y_{k-1}$ and $y_{k-1}^T y_{k-1}$ tend to zero, so the Nocedal scaling (3.6) becomes the ratio of two vanishing quantities, which in floating-point arithmetic may be dominated by rounding errors. In exact arithmetic on our quadratic γ_k^{N} is bounded, since $y_{k-1} = H s_{k-1}$ gives $\gamma_k^{\mathrm{N}} \in [1/\lambda_{\max}(H),\, 1/\lambda_{\min}(H)]$; on general problems, or when its computed value is corrupted by noise, an unbounded scaling factor would make the search direction unreliable. To prevent this, we clip the scaling factor to a finite interval:

$$\gamma_k^{\mathrm{safe}} = \mathrm{clip}\big(\gamma_k^{\mathrm{N}},\ \gamma_{\min},\ \gamma_{\max}\big)\,, \tag{3.8}$$

with defaults $\gamma_{\min} = 10^{-10}$ and $\gamma_{\max} = 10^{10}$. This retains the BB2 estimate in the well-behaved regime while preventing numerical overflow in degenerate cases.

Initialisation. All three strategies require at least one stored pair. At the first iteration, and after any memory reset, no curvature information is available and we set

$$\gamma_k = \frac{1}{\|\nabla f_k\|}, \tag{3.9}$$

so that the first direction is a steepest-descent step of unit length. This only affects the initial scale: from the second iteration onwards the chosen strategy takes over.

3.2.2 Two-Loop Recursion

The product $r = H_k \nabla f_k$ is computed by Algorithm 1 [1, Algorithm 9.1]. The algorithm implements the BFGS recurrence (3.3) unrolled over the last $\bar{m}$ pairs, applied implicitly to the gradient vector q. The *backward loop* computes the α_i scalars that will be needed later, while simultaneously applying the V_i^T factors to q from right to left. The *scaling step* applies the base approximation $H_k^0 = \gamma_k I$. The *forward loop* then applies the rank-one corrections $\rho_i s_i s_i^T$, using the α_i computed in the backward pass. Together, the two loops implement exactly the product $H_k q$ without ever constructing the matrix.

Algorithm 1 L-BFGS Two-Loop Recursion [1, Algorithm 9.1]

Require: Gradient $q \leftarrow \nabla f_k$; stored pairs $\{s_i, y_i\}_{i=k-\bar{m}}^{k-1}$; scaling γ_k
Ensure: $r = H_k \nabla f_k$

1: **for** $i = k-1,\, k-2,\, \ldots,\, k-\bar{m}$ **do**
2: $\quad \rho_i \leftarrow 1/(y_i^T s_i)$; $\quad \alpha_i \leftarrow \rho_i\, s_i^T q$; $\quad q \leftarrow q - \alpha_i\, y_i$ ▷ Apply V_i^T to q
3: **end for**
4: $r \leftarrow \gamma_k\, q$ ▷ Apply $H_k^0 = \gamma_k I$
5: **for** $i = k-\bar{m},\, \ldots,\, k-1$ **do**
6: $\quad \beta \leftarrow \rho_i\, y_i^T r$; $\quad r \leftarrow r + s_i\,(\alpha_i - \beta)$ ▷ Apply rank-one correction
7: **end for**
8: **return** r

Each of the $2\bar{m}$ loop iterations performs one dot product and one vector update (axpy) of length m, each costing $2m$ flops, and the scaling step costs m multiplications, for a total of $(8\bar{m}+1)\,m$ floating-point operations (counting one flop per addition or multiplication). This is a dramatic improvement over the $O(m^2)$ cost of storing and applying the full BFGS matrix.

3.2.3 Curvature Safeguard and Optional Restart

For the BFGS formula (3.3) to produce a positive definite update, the *curvature condition* $y_k^T s_k > 0$ must hold. To see why, note that $\rho_k = 1/(y_k^T s_k)$ appears in the rank-one term $\rho_k s_k s_k^T$: if $y_k^T s_k \le 0$, then $\rho_k \le 0$ and the update would subtract from H_k, potentially making H_{k+1} indefinite and the next direction p_{k+1} a direction of ascent. On our quadratic the condition always holds in exact arithmetic, since $y_k = Hs_k$ gives $y_k^T s_k = s_k^T H s_k \ge \lambda^2 \|s_k\|^2 > 0$; in floating-point arithmetic, however, a pair computed from very small steps may be dominated by rounding errors. We implement three levels of protection.

Scale-invariant curvature test. A new pair is stored only if

$$y_k^T s_k > \epsilon_{\mathrm{c}} \, \|s_k\| \, \|y_k\| , \qquad \epsilon_{\mathrm{c}} = 10^{-10}, \tag{3.10}$$

i.e. if the angle between s_k and y_k is not too close to $90°$; otherwise the pair is skipped and the memory is left unchanged. The test is *relative* on purpose. An absolute threshold, such as $y_k^T s_k > 10^{-10}$, is not invariant under scaling: since $y_k^T s_k = s_k^T H s_k$ grows as $\|s_k\|^2$, it rejects perfectly valid pairs once the steps become small near the solution. The memory then stops being refilled, and the method degenerates into a scaled gradient method for the rest of the run. The relative test (3.10) depends only on the direction of the pair, not on its size. With the exact line search it never rejects a pair on our problem; it fails only once, with the Wolfe line search near stagnation, where the computed $y_k^T s_k$ is exactly zero (Section 4.2.1).

Negative-curvature reset. If $y_k^T s_k \le 0$, the curvature condition is violated: the entire stored memory is cleared and the next direction is computed from $H_k^0 = \gamma_k I$ alone. This guarantees that every stored pair has $\rho_k > 0$, as required by the BFGS update (3.3).

Curvature-based restart. As an optional safeguard, disabled by default, the memory can also be cleared when $y_k^T s_k > 0$ but the curvature estimate captured by the new pair changes dramatically relative to the previous one. To detect this, we track the quantity

$$\theta_k = \frac{y_k^T s_k}{y_k^T y_k}, \tag{3.11}$$

which has the same form as the Nocedal scaling (3.6): it estimates the effective inverse-Hessian scale along s_k. A sudden drop,

$$\theta_k < \xi \, \theta_{k-1}, \qquad \xi \in (0,1), \tag{3.12}$$

signals that the curvature along the new step is much smaller than along the previous one. This may happen when the iterates move into a qualitatively different region of the landscape: the stored pairs were computed under a different curvature regime and may give a misleading approximation of H_k. When the restart is enabled, the threshold is $\xi = 0.2$.

3.3 Line Search Strategies

3.3.1 Strong Wolfe Conditions

Once the descent direction p_k is computed, the step length α_k must be chosen to make sufficient progress without overshoot. The standard criterion for quasi-Newton methods is the *strong Wolfe conditions* [1, Eq. 3.7]:

$$f(w_k + \alpha_k p_k) \leq f(w_k) + c_1\, \alpha_k\, \nabla f_k^T p_k, \tag{3.13}$$

$$\left|\nabla f(w_k + \alpha_k p_k)^T p_k\right| \leq c_2\, \left|\nabla f_k^T p_k\right|. \tag{3.14}$$

Condition (3.13) (Armijo) ensures sufficient decrease: the function must drop by at least a fraction c_1 of the decrease predicted by the directional derivative, which rules out steps that are too long. Condition (3.14) prevents the step from being too short: the directional derivative at the new point must be small enough in absolute value, which ensures that the new pair (s_k, y_k) carries meaningful curvature information (in particular, it guarantees $y_k^T s_k > 0$). Following standard practice for quasi-Newton methods [1], we set $c_1 = 10^{-4}$ and $c_2 = 0.9$. Our implementation uses a bracketing-and-zoom procedure with cubic interpolation [1, Section 3.4].

3.3.2 Exact Line Search for Quadratic Objectives

The Wolfe conditions are designed for general nonlinear objectives and accept a range of step lengths rather than requiring the exact minimiser. For our quadratic f, we can do much better. The restriction of f along any direction p is a one-dimensional parabola:

$$\varphi(\alpha) = f(w + \alpha\, p) = f(w) + \alpha\, \nabla f(w)^T p + \frac{\alpha^2}{2}\, p^T H p. \tag{3.15}$$

Since $H \succ 0$, the coefficient $p^T H p > 0$ for all $p \neq 0$, so φ is a strictly convex parabola with a unique global minimum at $\varphi'(\alpha^*) = 0$, i.e.:

$$\alpha^* = -\frac{\nabla f(w)^T p}{p^T H p}. \tag{3.16}$$

For our specific Hessian $H = XX^T + \lambda^2 I_m$, the denominator can be expanded without forming H explicitly:

$$p^T H p = p^T (XX^T + \lambda^2 I_m)\, p = \left\|X^T p\right\|^2 + \lambda^2 \left\|p\right\|^2, \tag{3.17}$$

so the complete formula becomes a generalisation of [1, Eq. 3.25] to an arbitrary descent direction:

$$\alpha^* = -\frac{\nabla f(w)^T p}{\left\|X^T p\right\|^2 + \lambda^2 \left\|p\right\|^2}. \tag{3.18}$$

The only non-trivial cost is the single matrix-vector product $X^T p \in O(mn)$, about half the cost of one gradient evaluation, and far cheaper than the multiple function and gradient evaluations required by the Wolfe bracketing procedure. Since α^* is the *exact* minimiser of φ, it fully exploits the parabolic structure and yields the best possible step at each iteration, reducing the number of iterations required to reach a given accuracy compared to the Wolfe variant.

3.4 Complete Algorithm

Algorithm 2 Enhanced L-BFGS [1, Algorithm 9.2]

Require: Starting point w_0; memory $\bar{m}$; tolerance ε; scaling strategy $\in$ {BB2, BB1, safe}; restart flag (default off); restart threshold ξ; curvature tolerance ϵ_c

$k \leftarrow 0$; compute ∇f_0; $\tau \leftarrow \varepsilon \|\nabla f_0\|$; $\theta_{\text{prev}} \leftarrow 1$
while $\|\nabla f_k\| > \tau$ **do**
  if $k > 0$ **and** $\alpha_{k-1} \approx 0$ **and** $|f_k - f_{k-1}|/|f_k| < \varepsilon_{\text{mach}}$ **then**
    stop ▷ Numerical stagnation, Section 3.5
  end if
  if no pairs are stored **then**
    $\gamma_k \leftarrow 1/\|\nabla f_k\|$ ▷ No curvature information yet, Eq. (3.9)
  else
    Compute γ_k via chosen scaling strategy (Eq. (3.6), (3.7), or (3.8))
  end if
  $p_k \leftarrow -H_k \nabla f_k$ ▷ Via Algorithm 1 with $H_k^0 = \gamma_k I$
  if $\nabla f_k^T p_k \geq 0$ **then**
    $p_k \leftarrow -\nabla f_k$ ▷ Descent safeguard: fall back to steepest descent
  end if
  Compute α_k via exact LS (3.18) or Wolfe conditions (3.13)–(3.14)
  $w_{k+1} \leftarrow w_k + \alpha_k p_k$; $s_k \leftarrow \alpha_k p_k$; $y_k \leftarrow \nabla f_{k+1} - \nabla f_k$
  if $y_k^T s_k > \epsilon_c \|s_k\| \|y_k\|$ **then** ▷ Scale-invariant test, Eq. (3.10)
    $\theta_k \leftarrow (y_k^T s_k)/(y_k^T y_k)$
    if restart enabled **and** $\theta_k < \xi\, \theta_{\text{prev}}$ **then**
      Clear all stored pairs ▷ Curvature-based restart, Eq. (3.12)
    end if
    Store (s_k, y_k); discard oldest if #stored $> \bar{m}$; $\theta_{\text{prev}} \leftarrow \theta_k$
  else if $y_k^T s_k \leq 0$ **then**
    Clear all stored pairs ▷ Negative-curvature reset
  else
    Discard (s_k, y_k) ▷ Nearly orthogonal pair; skipped
  end if
  $k \leftarrow k + 1$
end while
return w_k

3.5 Stopping Criterion

The primary stopping criterion uses a *relative* gradient tolerance:

$$\|\nabla f(w_k)\| \leq \varepsilon \cdot \|\nabla f(w_0)\|. \tag{3.19}$$

The relative form is preferred over an absolute threshold $\|\nabla f(w_k)\| \leq \varepsilon$ because it is scale-invariant: the magnitude of ∇f depends on the problem scale (and in particular on λ), so an absolute threshold that is appropriate for one value of λ may be either too loose or too tight for another. Dividing by $\|\nabla f(w_0)\|$ normalizes for this effect.

As a secondary safeguard, the algorithm detects *numerical stagnation*: if the relative change $|f_k - f_{k-1}|/|f_k|$ drops below machine epsilon (approximately 10^{-16}) while the step length $\alpha_k \approx 0$, further progress is impossible due to floating-point limitations, and the iteration is terminated early.

3.6 Convergence Analysis

3.6.1 Global Linear Convergence

The convergence of L-BFGS on our problem follows from [3, Theorem 7.1]. That theorem requires three conditions on f, which we verify directly using the properties established in Section 2:

1. $f \in C^2$: satisfied, since f is quadratic;
2. the sublevel set $\mathcal{L}_0 = \{w : f(w) \le f(w_0)\}$ is convex: satisfied, since f is μ-strongly convex with $\mu = \lambda^2 > 0$, so all sublevel sets are ellipsoids;
3. there exist $M_1, M_2 > 0$ such that $M_1\|v\|^2 \le v^T\nabla^2 f(w)\, v \le M_2\|v\|^2$ for all v: satisfied with $M_1 = \mu = \lambda^2$ and $M_2 = L = \lambda_{\max}(XX^T) + \lambda^2$, since $\nabla^2 f = H$ is constant.

The theorem further requires the initial matrices H_k^0 and their inverses to be uniformly bounded, and the step lengths to satisfy the Wolfe conditions. Both hold here: since $y_k = Hs_k$ on our quadratic, both the BB2 and BB1 scalings are Rayleigh-type quotients of H, so $\gamma_k \in [1/\lambda_{\max}(H),\, 1/\lambda_{\min}(H)]$; and the exact step (3.18) satisfies the Wolfe conditions, since $\varphi'(\alpha^*) = 0$ and, for $c_1 \le 1/2$, the decrease of the parabola at its minimiser exceeds the Armijo threshold. Finally, the theorem allows the number of stored pairs to vary between iterations, which covers the memory resets of Section 3.2.3. Under these conditions, the iterates satisfy the linear convergence bound:

$$f(w_k) - f(w^*) \le r^k\,(f(w_0) - f(w^*)), \quad r \in [0, 1), \tag{3.20}$$

where the rate r depends on the condition number $\kappa(\hat{X})$: the more ill-conditioned the problem, the slower the linear rate. This is a worst-case bound for general strongly convex functions; on our quadratic the stronger result of Section 3.6.3 applies.

3.6.2 Superlinear Convergence

For full BFGS (i.e. $\bar{m}$ large enough to store the entire history), [1, Theorem 8.6] guarantees superlinear convergence on strongly convex functions: the error ratio

$$\frac{\|w_{k+1} - w^*\|}{\|w_k - w^*\|} \longrightarrow 0. \tag{3.21}$$

Intuitively, as H_k accumulates curvature information, its action along the search directions becomes increasingly accurate (the Dennis–Moré condition), so that $p_k = -H_k\nabla f_k$ becomes increasingly close to the Newton step $-H^{-1}\nabla f_k$, even though H_k itself need not converge to H^{-1}.

3.6.3 Finite Termination on the Quadratic

On our quadratic, a stronger result holds in exact arithmetic. When a method of the Broyden class, which includes BFGS, is applied with exact line searches to a strongly convex quadratic, it converges in at most as many iterations as the dimension of the problem, and with initial matrix $B_0 = I$ it generates the same iterates as the conjugate gradient method (CG); with a general B_0, those of CG preconditioned by B_0 [1, Theorem 8.4]. CG, in turn, terminates in at most r iterations when the Hessian has only r distinct eigenvalues [1, Theorem 5.4].

In our problem the relevant number is n, not m. Starting from $w_0 = 0$, every gradient $\nabla f_k = Hw_k - Xy$ lies in the range of X, and so do all the iterates. On this n-dimensional subspace the Hessian has eigenvalues $\sigma_i^2(X) + \lambda^2$, $i = 1, \ldots, n$: at most $n = 12$ distinct values, whatever λ is. In exact arithmetic, CG and BFGS with exact line search therefore terminate in at most n iterations, independently of the conditioning. L-BFGS with $\bar{m} \geq n$ stores all the pairs generated before termination and differs from this setting only in that its initial matrix $H_k^0 = \gamma_k I$ is rescaled at every iteration; the theorem does not cover this variant directly, but suggests the behaviour observed in Chapter 4.

Chapter 4

L-BFGS Experimental Results

All experiments use the matrix $X \in \mathbb{R}^{m \times n}$ with $m = 500$ samples and $n = 12$ features after standard preprocessing (zero mean, unit variance per feature). The response vector $y \in \mathbb{R}^n$ is drawn from a standard normal distribution with a fixed random seed (`seed` $= 42$). The reference solution w^* is computed from the thin SVD of X, $w^* = U \operatorname{diag}\big(\sigma_i/(\sigma_i^2 + \lambda^2)\big)V^T y$, which does not form XX^T: the normal equations lose accuracy as λ decreases (Section 6.2) and, on this problem, are less accurate than L-BFGS itself (Section 4.2.4).

Unless stated otherwise, we use the default configuration selected by a grid search over the algorithm's hyperparameters on the base problem (with the initial point fixed to $w_0 = 0$, the canonical neutral choice): regularization $\lambda = 0.5$ (yielding $\kappa(\hat{X}) = 151.49$), exact line search, memory size $\bar{m} = 12$, equal to the rank n of XX^T, the Barzilai–Borwein H_0 scaling (`bb1`), and no curvature-based restart. The grid search shows that, from $\bar{m} = 12$ upwards, the iteration count is insensitive to the H_0 scaling and to the restart; these choices only matter below the effective rank, where `bb1` without restart is the most robust option (Sections 4.2.5 and 4.2.6). The relative tolerance is $\varepsilon = 10^{-14}$. All experiments are run on a single machine; wall-clock measurements use `time.perf_counter()` and report the median over ten repetitions after a discarded warm-up run, while FLOP counts use the operation model of Section 4.3.2.

The experimental discussion is organised in three parts. We first verify the *theoretical convergence properties* of Section 3.6 (convergence rates and finite termination, Newton-direction alignment, and a comparison against steepest descent and conjugate gradient). We then examine the *algorithmic design choices* of our L-BFGS variant (line search, Wolfe constants, memory size, regularization, H_0 scaling, restart). Finally, we probe the *numerical and computational behaviour* of the method (sensitivity to initialisation, theoretical and actual cost, and scalability in n and m).

4.1 Theoretical Convergence Properties

4.1.1 Empirical Convergence Rates and Finite Termination

We track the convergence behaviour at three memory sizes: $\bar{m} = 3$ (below the effective rank), $\bar{m} = 12$ (the default), and $\bar{m} = m = 500$ (effectively full BFGS, since the algorithm always converges in fewer than m iterations). We record two per-iteration quantities: the Q-linear ratio $r_k^{\text{lin}} = (f_k - f^*)/(f_{k-1} - f^*)$, which the linear bound of Eq. (3.20) keeps below a constant smaller than 1, and the Q-superlinear ratio $r_k^{\text{sup}} = \|w_{k+1} - w^*\| \,/\, \|w_k - w^*\|$, whose decay towards 0 is the signature of the superlinear regime of Eq. (3.21).

Table 4.1: L-BFGS at three memory sizes ($\lambda = 0.5$, exact LS, $\varepsilon = 10^{-14}$).

$\bar{m}$	**Iterations**	$\|w - w^*\|$	$\|f - f^*\|$
3	15	2.19×10^{-15}	1.4×10^{-17}
12	12	2.20×10^{-15}	6.9×10^{-17}
500	12	2.20×10^{-15}	6.9×10^{-17}

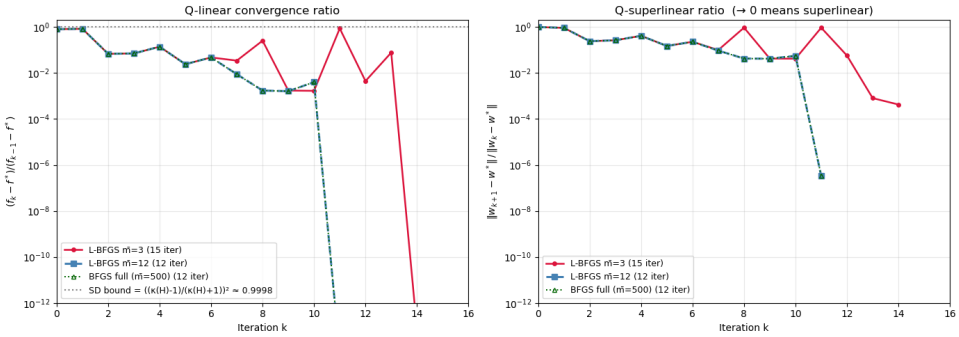


Figure 4.1: Q-linear (left) and Q-superlinear (right) convergence ratios ($\lambda = 0.5$, exact LS, $\varepsilon = 10^{-14}$). The dotted grey line in the left panel marks the steepest-descent bound $r_{\mathrm{SD}} \approx 0.9998$.

The two larger memories, $\bar{m} = 12$ and $\bar{m} = 500$, produce *identical* trajectories and converge in 12 iterations to an accuracy of 2.2×10^{-15}. After the first iterations both ratios fluctuate with a decreasing trend: the Q-linear one between about 10^{-1} and 10^{-3}, far below the steepest-descent bound $r_{\mathrm{SD}} = \big((\kappa(H) - 1)/(\kappa(H) + 1)\big)^2 \approx 0.9998$, with $\kappa(H) = \kappa(\hat{X})^2$, which is essentially 1 on this problem; the Q-superlinear one between about 0.3 and 0.05, without approaching zero. The run then ends with an abrupt collapse of both ratios at the last iteration, rather than a gradual superlinear decay. This is the signature of the finite termination of Section 3.6.3: the Hessian restricted to the range of X, where the iterates live, has 12 distinct eigenvalues, and once the method has explored the corresponding 12 directions the solution is reached to machine precision. The values of $|f - f^*|$, of order 10^{-17}, are at the rounding level $\varepsilon_{\mathrm{mach}} f^*$ of the objective ($f^* \approx 0.09$); the computed differences are in fact slightly negative. The last point of the left panel is therefore produced by flooring this difference at a small positive constant before taking the ratio, so its vertical position carries no information: only the collapse itself does. In the right panel, where $\|w_k - w^*\|$ stays positive throughout, the drop of about five orders of magnitude at the final step is measured rather than imposed.
With the memory below the effective rank, $\bar{m} = 3$ follows the same trajectory for the first iterations and then needs three more iterations (15), reaching the same accuracy. Its ratios show isolated returns close to or slightly above 1, i.e. steps of little progress, or steps that temporarily move the iterate away from w^*, where the three stored pairs cannot represent the curvature directions explored earlier. A memory comparable to the effective rank therefore matches the full-memory method *exactly*: by Lemma 3, XX^T has only $n = 12$ non-zero eigenvalues, so storing 12 vectors is as good as storing all 500.

4.1.2 Comparison with Steepest Descent and Conjugate Gradient

To assess the practical value of the curvature information used by L-BFGS, we compare it with two natural baselines: *steepest descent* (SD) with exact line search, and the linear *conjugate gradient* method (CG) applied to the symmetric positive definite (SPD) system $Hw = Xy$ through implicit

products $Hv = X(X^T v) + \lambda^2 v$, with the same per-iteration cost as L-BFGS. All methods start from $w_0 = 0$ with $\varepsilon = 10^{-14}$, in a well-conditioned ($\lambda = 1$, $\kappa \approx 76$) and an ill-conditioned ($\lambda = 10^{-4}$, $\kappa \approx 7.6 \times 10^5$) regime.

Table 4.2: Iteration counts and final gradient norms across four methods ($\varepsilon = 10^{-14}$, exact LS where applicable).

	Well-cond. ($\lambda = 1$)		*Ill-cond.* ($\lambda = 10^{-4}$)	
Method	iter	final $\|\nabla f\|$	iter	final $\|\nabla f\|$
Steepest descent	1447	6.14×10^{-13}	1725	6.55×10^{-13}
Conjugate gradient	15	5.47×10^{-15}	15	1.90×10^{-14}
L-BFGS ($\bar{m} = 12$)	12	5.98×10^{-14}	12	6.61×10^{-14}
Full BFGS ($\bar{m} = 500$)	12	5.98×10^{-14}	12	6.61×10^{-14}

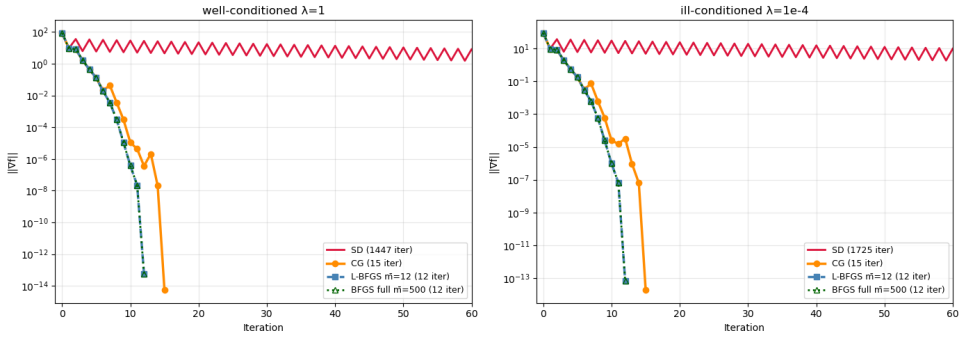


Figure 4.2: Convergence of the four methods ($\|\nabla f\|$ vs. iteration) in the well-conditioned (left) and ill-conditioned (right) regimes. L-BFGS and full BFGS overlap; CG follows them for the first iterations and then lags behind. The SD curve is truncated at 60 iterations; its full length is reported in Table 4.2.

Steepest descent is uncompetitive: using only the gradient, it needs over 1400 iterations in both regimes, zigzagging across the elongated level sets; its iteration count grows with κ, from 1447 to 1725. L-BFGS instead needs 12 iterations in *both* regimes, for $\bar{m} = 12$ and full BFGS alike: its iteration count does not depend on κ, in agreement with the finite-termination argument of Section 3.6.3.
In exact arithmetic CG would terminate after the same 12 steps, since the Hessian restricted to the range of X has 12 distinct eigenvalues [1, Theorem 5.4]. In practice it needs 15, and checking the *true* gradient at each iterate, rather than the residual updated by recurrence, gives exactly the same counts for every n: the difference is not due to the stopping test. Figure 4.2 shows what happens: CG and L-BFGS produce the same gradient norms for the first iterations, as expected from their close relation on quadratics (Section 3.6.3), and then CG drifts away. Rounding errors make the CG directions progressively lose conjugacy and delay convergence, whereas L-BFGS stores the curvature pairs explicitly and reaches the theoretical count. The final gradient norm of CG is lower than that of L-BFGS, but this reflects the three extra iterations it performs rather than a more accurate solution. On non-quadratic objectives, where CG loses its conjugacy guarantees altogether, the advantage of L-BFGS would be even larger.

4.1.3 Alignment with the Newton Direction

To explain the behaviour of the convergence ratios, we measure how closely the L-BFGS direction $p_k = -H_k \nabla f_k$ approximates the Newton direction $p_k^{\mathrm{N}} = -H^{-1}\nabla f_k$ through the alignment defect $1 - \cos\theta_k$, with $\cos\theta_k = \langle p_k, p_k^{\mathrm{N}} \rangle / (\|p_k\| \, \|p_k^{\mathrm{N}}\|)$. The Newton direction is computed from a cached Cholesky factorisation of H and used *only as a measurement oracle*, not by the algorithm. The series are truncated once $\|\nabla f_k\| < 10^{-9}$, below which both directions are computed on numerical noise.

Table 4.3: Alignment defect $1 - \cos\theta_k$ at the last *clean* iteration ($\|\nabla f_k\| \geq 10^{-9}$), $\lambda = 0.5$, exact LS.

$\bar{m}$	**Iterations**	**Clean iterations**	**Final** $1 - \cos\theta_k$
3	15	15	2.50×10^{-8}
12	12	12	2.55×10^{-14}
500	12	12	2.55×10^{-14}

With $\bar{m} = 12$ and full BFGS the defect falls to 2.6×10^{-14}, i.e. to machine precision: by the end of the run the L-BFGS direction *is* the Newton direction, which is why the last step reaches the solution. Even $\bar{m} = 3$ aligns closely, to 2.5×10^{-8}: a memory below the effective rank limits how accurately H_k can reproduce H^{-1}, which explains the few extra iterations, but the direction remains strongly Newton-like. The quality of the Newton approximation built by the two-loop recursion thus grows with the memory and saturates once the memory reaches the effective rank.

4.2 Algorithmic Design Choices

We now examine the design choices of our implementation: the line search, the Wolfe constants, the memory size, the regularization parameter, the H_0 scaling and the curvature-based restart. Each experiment varies one choice and keeps the others at their default value.

4.2.1 Exact Line Search vs. Strong Wolfe Line Search

We compare the two line-search strategies with the standard Wolfe constants $(c_1, c_2) = (10^{-4}, 0.9)$ recommended by Nocedal and Wright [1, Sec. 3.1] for quasi-Newton methods.

Table 4.4: Comparison of line-search strategies ($m = 500$, $n = 12$, $\lambda = 0.5$, $\bar{m} = 12$, $\varepsilon = 10^{-14}$).

Line Search	**Iterations**	$\|w - w^*\|$	**Time (ms)**	**Stop reason**
Exact	12	2.20×10^{-15}	0.44	tolerance reached
Wolfe ($c_1 = 10^{-4}, c_2 = 0.9$)	30	1.43×10^{-9}	1.73	stagnation

Exact line search dominates: it converges in 12 iterations against 30, reaches an accuracy six orders of magnitude higher, and is about 4× faster in wall-clock time. Its closed-form step, Eq. (3.18), exploits the quadratic structure exactly at the cost of a single matrix-vector product, whereas each Wolfe iteration needs several function and gradient evaluations.

The Wolfe variant does not diverge, but stops on stagnation at $\|\nabla f\| \approx 2 \times 10^{-8}$, without reaching the tolerance. This is an intrinsic limit of line searches based on function values: with such a small gradient, the decrease of f required by the Armijo condition (3.13) falls below the precision with which $f \approx 0.09$ can be computed, about $\varepsilon_{\mathrm{mach}} f \approx 2 \times 10^{-17}$, so the line search can no longer verify

it. One iteration before stopping, the run also triggers the negative-curvature reset of Section 3.2.3: the computed $y_k^T s_k$ is exactly zero, so the pair carries no usable curvature and the memory is cleared. Both symptoms have the same origin, the loss of significance in quantities derived from f near the solution. The exact line search never compares function values and has no such limit.

4.2.2 Sensitivity to the Wolfe Constants c_1, c_2

We sweep $c_1 \in \{10^{-4}, 10^{-3}, 10^{-2}\}$ and $c_2 \in \{0.1, 0.2, \ldots, 0.9\}$ in the default configuration. Table 4.5 reports the results for $c_1 = 10^{-4}$.

Table 4.5: Effect of the Wolfe constants on convergence ($\lambda = 0.5$, $\bar{m} = 12$, $\varepsilon = 10^{-14}$, $c_1 = 10^{-4}$; the other values of c_1 give identical results).

c_2	**Iterations**	**Total LS evals**	**Mean LS evals**	$\|w - w^*\|$
0.1	16	91	5.69	4.83×10^{-9}
0.2	24	139	5.79	5.44×10^{-10}
0.3	30	154	5.13	2.21×10^{-9}
0.4	26	105	4.04	1.12×10^{-9}
0.5	29	106	3.66	1.46×10^{-10}
0.6	29	95	3.28	1.46×10^{-10}
0.7	29	98	3.38	4.57×10^{-10}
0.8	30	111	3.70	1.13×10^{-9}
0.9	30	120	4.00	1.43×10^{-9}

The constant c_1 has no effect: all three values give identical results, because the Armijo condition is already satisfied at the trial step selected by the curvature condition. The curvature constant c_2 matters more, and strict values work best: $c_2 = 0.1$ needs 16 iterations, the closest to exact line search (12), while all values from 0.3 to 0.9 need 26–30. A small c_2 forces the accepted step close to the minimiser along p_k, i.e. close to the exact step. It costs more line-search evaluations per iteration (5.7 on average at $c_2 = 0.1$, against 3.3–4.0 for $c_2 \geq 0.5$), but needs fewer iterations, so its total number of evaluations is still the lowest (91). In all cases the final accuracy is limited by the stagnation of Section 4.2.1, to 10^{-10}–10^{-9}. The standard value $c_2 = 0.9$, designed for general quasi-Newton problems, is therefore not the best choice on this quadratic, and exact line search remains preferable whenever it is available in closed form.

4.2.3 Effect of the Memory Parameter $\bar{m}$

We vary $\bar{m} \in \{1, 2, 3, 5, 10, 12, 20, 40\}$ in the default configuration.

Table 4.6: Effect of memory size $\bar{m}$ on L-BFGS convergence ($m = 500$, $n = 12$, $\lambda = 0.5$, exact LS, $\varepsilon = 10^{-14}$).

$\bar{m}$	**Iterations**	$\|w - w^*\|$	**Time (ms)**
1	20	5.06×10^{-14}	0.49
2	17	3.45×10^{-15}	0.44
3	15	2.19×10^{-15}	0.41
5	14	2.09×10^{-15}	0.43
10	13	2.22×10^{-15}	0.47
12	12	2.20×10^{-15}	0.42
20	12	2.20×10^{-15}	0.42
40	12	2.20×10^{-15}	0.41

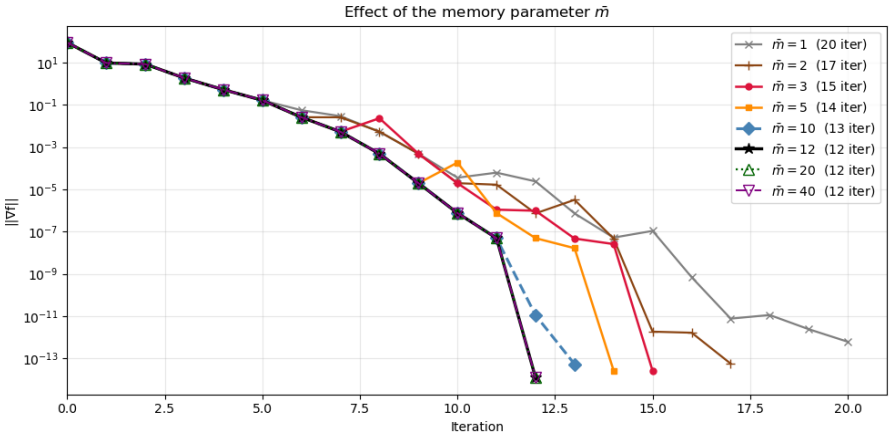


Figure 4.3: Gradient-norm convergence for $\bar{m} \in \{1, 2, 3, 5, 10, 12, 20, 40\}$ (log scale on the y-axis, iteration counts in the legend). All memories, even $\bar{m} = 1$, follow the same trajectory in the first iterations; the curves for $\bar{m} \geq 12$ coincide and terminate after 12 iterations.

The iteration count decreases *monotonically* with the memory, from 20 iterations with a single pair to 12 at $\bar{m} = 12$, and then saturates. The saturation point is exactly the effective rank $n = 12$: once the stored pairs span the curvature directions of the Hessian restricted to the range of X, additional pairs carry no new information, consistently with Sections 4.1.1 and 4.1.3. Figure 4.3 shows how this happens: all memories, even $\bar{m} = 1$, follow the same trajectory for the first iterations, and the smaller memories leave it one after another, between iterations 7 and 12 — the smaller the memory, the earlier the departure. This agrees with the classical result that memoryless BFGS, i.e. a single stored pair with $H_0 = I$, combined with an exact line search coincides with the Hestenes–Stiefel conjugate gradient method [1, Sec. 9.1]: in exact arithmetic even a minimal memory would follow the CG trajectory. In practice, rounding errors and the rescaling of $H_k^0 = \gamma_k I$ at every iteration break this equivalence, and a small memory, which relies on the implicit conjugacy of the directions rather than on stored curvature pairs, is affected first.

Even a very small memory converges reliably, and to the same accuracy, of order 10^{-15} (slightly higher, 5×10^{-14}, with $\bar{m} = 1$). This is consistent with the observation of Nocedal and Wright [1, Sec. 9.1] that modest memories often give satisfactory results. The wall-clock time stays between 0.41 and 0.49 ms for every memory size: on this problem the differences of a few iterations are comparable to the measurement noise. The practical recommendation is to set $\bar{m}$ equal to the effective rank: smaller

values cost a few extra iterations, larger values only add storage.

4.2.4 Effect of the Regularization Parameter λ

We study convergence across five regularization regimes. Recall from Eq. (2.7) that $\kappa(\hat{X})$ scales as $O(1/\lambda)$ for small λ, while large λ drives $\kappa(\hat{X}) \to 1$, making the problem nearly spherical. The last column of Table 4.7 reports the distance between the normal-equations solution and the SVD reference.

Table 4.7: Effect of λ on condition number and convergence ($m = 500$, $n = 12$, exact LS, $\bar{m} = 12$, $\varepsilon = 10^{-14}$; w^* from the SVD reference).

λ	$\kappa(\hat{X})$	**Iter**	$\|w - w^*\|$	t_{LBFGS} (ms)	$\|w_{\text{NE}} - w^*\|$
10^{-4}	7.6×10^5	12	2.23×10^{-15}	0.42	1.11×10^{-5}
10^{-2}	7.6×10^3	12	2.26×10^{-15}	0.41	1.11×10^{-9}
1	75.8	12	1.96×10^{-15}	0.41	1.03×10^{-13}
10^2	1.3	6	1.48×10^{-17}	0.16	1.52×10^{-17}
10^4	1.0	3	2.51×10^{-21}	0.07	2.51×10^{-21}

Neither the iteration count nor the accuracy of L-BFGS depends on the conditioning. For every $\lambda \leq 1$, from $\kappa \approx 76$ to $\kappa \approx 7.6 \times 10^5$, L-BFGS needs exactly 12 iterations and reaches $\|w - w^*\| \approx 2 \times 10^{-15}$. Both facts have the same structural reason. Starting from $w_0 = 0$, all iterates remain in the range of X, where the Hessian has the n eigenvalues $\sigma_i^2 + \lambda^2$: there are 12 of them whatever λ is, which fixes the iteration count (Section 3.6.3), and their ratio $(\sigma_{\max}^2 + \lambda^2)/(\sigma_{\min}^2 + \lambda^2)$ stays bounded as $\lambda \to 0$, because $\sigma_{\min}(X) > 0$, which preserves the accuracy. L-BFGS never forms XX^T and never sees the full condition number.

The normal equations behave very differently: their distance from the SVD reference grows from 10^{-13} at $\lambda = 1$ to 1.1×10^{-5} at $\lambda = 10^{-4}$, following the squared condition number of Eq. (2.6) (Section 6.2). At small λ L-BFGS is therefore far more accurate than a reference computed from the normal equations, which is why all experiments use the SVD reference.

In the regime $\lambda \gg \sigma_{\max}(X)$ the regularization term dominates, $H \approx \lambda^2 I$, and L-BFGS converges in 3–6 iterations on a nearly spherical problem. Across all regimes L-BFGS runs in under 0.5 ms, and the wall-clock time follows the iteration count.

This experiment also clarifies why we adopt $\lambda = 0.5$ as the default. The parameter λ is not an algorithmic knob but part of the problem definition: it sets the trade-off between fitting the data and controlling $\|w\|$. Although large λ makes the problem numerically trivial, the resulting solution is dominated by the penalty term and is essentially $w \approx 0$, ignoring the data. The value $\lambda = 0.5$ yields a moderately ill-conditioned instance ($\kappa \approx 151$) that is representative of realistic regularization.

4.2.5 H_0 Scaling Strategies

We compare the three strategies of Section 3.2.1 for the initial Hessian approximation $H_k^0 = \gamma_k I$: the Nocedal scaling (BB2, Eq. (3.6)), the Barzilai–Borwein companion (BB1, Eq. (3.7)), and the safeguarded variant. We run each strategy at the default memory $\bar{m} = 12$ and at $\bar{m} = 4$, below the effective rank, with and without the restart, in both conditioning regimes. Here and in Section 4.2.6 the ill-conditioned regime is realised with $\lambda = 10^{-2}$ ($\kappa \approx 7.6 \times 10^3$), a milder value than the $\lambda = 10^{-4}$ of Section 4.1.2; since the iteration count of L-BFGS does not depend on κ (Section 4.2.4), this choice does not affect the comparison between strategies.

Table 4.8: Iterations for each H_0 scaling, without / with the curvature-based restart, and median time without restart ($m = 500$, $n = 12$, exact LS, $\varepsilon = 10^{-14}$).

		Well-cond. ($\lambda = 1$)		*Ill-cond.* ($\lambda = 10^{-2}$)	
$\bar{m}$	**Strategy**	iter (off / on)	ms	iter (off / on)	ms
12	`nocedal`	12 / 12	0.42	12 / 12	0.41
12	`bb1`	12 / 12	0.42	12 / 12	0.42
12	`safeguarded`	12 / 12	0.44	12 / 12	0.44
4	`nocedal`	20 / 15	0.57	23 / 18	0.66
4	`bb1`	14 / 15	0.39	15 / 14	0.43
4	`safeguarded`	20 / 15	0.61	23 / 18	0.70

At the default memory the scaling does not matter: all three strategies need 12 iterations in both regimes, with and without the restart, and reach the same accuracy. Below the effective rank the picture changes. Without restart, `bb1` needs 14 and 15 iterations, against 20 and 23 for `nocedal`, and is about 1.5× faster in wall-clock time; it also reaches a higher accuracy (5×10^{-15} against 10^{-13}–3×10^{-14}). By the Cauchy–Schwarz inequality $\gamma^{\mathrm{BB1}} \geq \gamma^{\mathrm{N}}$: when the few stored pairs carry little curvature information, the larger initial scale produces more effective steps. This robustness is why `bb1` is the default. The restart helps `nocedal` at small memory (20 → 15 and 23 → 18 iterations) but brings no benefit to `bb1`, which already performs as well without it.
Finally, `nocedal` and `safeguarded` produce identical iterates in every configuration. The safeguard clips γ into a safe interval to guard against numerical degeneration near the solution, where it becomes the ratio of two vanishing quantities; on this quadratic γ never approaches the clipping bounds, so the safeguard never activates.

4.2.6 Curvature-Based Restart

The curvature-based restart of Section 3.2.3 clears the L-BFGS memory whenever the curvature ratio $\theta_k = (s^T y)/(y^T y)$ drops by more than a factor ξ relative to the previous stored pair (Eq. (3.12)). We test it ($\xi = 0.2$) against a no-restart control at the default memory $\bar{m} = 12$ and at the starved $\bar{m} = 3$, in both regimes.

Table 4.9: Effect of the curvature-based restart ($\xi = 0.2$) at two memory sizes ($m = 500$, $n = 12$, exact LS, `bb1`, $\varepsilon = 10^{-14}$).

		Well-cond. ($\lambda = 1$)		*Ill-cond.* ($\lambda = 10^{-2}$)	
$\bar{m}$	**Restart**	iter	restarts	iter	restarts
12	off	12	0	12	0
12	on	12	0	12	0
3	off	15	0	15	0
3	on	15	4	17	5

At the default memory the restart never fires: when enough pairs are stored, the curvature ratio never drops sharply on this quadratic, and the results are identical with and without it. At $\bar{m} = 3$ it fires four to five times, but without benefit: it leaves the well-conditioned case unchanged (15 iterations) and slows the ill-conditioned one (15 → 17), with a lower final accuracy (1.1×10^{-13}

against 3.2×10^{-15}). The discarded pairs still carried useful curvature information. Sweeping the threshold $\xi \in \{0.05, 0.10, 0.20, 0.40, 0.70\}$ at $\bar{m} = 3$ in the ill-conditioned case gives 17 iterations for every value, with 3 restarts at $\xi = 0.05$ and 5 for the others: the outcome is insensitive to the threshold. Since the restart is inactive at the default memory and does not help `bb1` at small memory, it is disabled by default.

4.3 Numerical and Computational Scalability

The previous sections characterised the convergence of L-BFGS on the default ML-CUP instance. We now probe three remaining practical aspects: the dependence on the starting point, the computational cost and storage, and the behaviour as the problem dimensions n and m change.

4.3.1 Sensitivity to the Initial Iterate w_0

Theory guarantees convergence from any starting point on a strongly convex quadratic. We test four starting points spanning four orders of magnitude in $\|w_0 - w^*\|$: the zero vector, two random Gaussian vectors with standard deviations 1 and 100, and a point near the optimum, $w^* + 0.01\,\mathcal{N}(0, I)$.

Table 4.10: Sensitivity to the initial iterate ($\lambda = 0.5$, $\bar{m} = 12$, exact LS, $\varepsilon = 10^{-14}$).

w_0	$\|w_0 - w^*\|$	**Iterations**	**Final** $\|w - w^*\|$
zeros	8.46×10^{-1}	12	2.20×10^{-15}
$\mathcal{N}(0, 1)$	2.20×10^{1}	13	8.31×10^{-15}
$\mathcal{N}(0, 100^2)$	2.19×10^{3}	13	9.25×10^{-13}
near optimum	2.27×10^{-1}	13	2.27×10^{-15}

The starting point does not matter: all four starts converge in 12 or 13 iterations, although $\|w_0 - w^*\|$ spans four orders of magnitude. Only the start $w_0 = 0$ keeps the iterates exactly in the range of X; the other starts have a component outside it, which costs at most one extra iteration. The farthest start stops at a slightly lower accuracy (9×10^{-13}) because the relative criterion $\varepsilon \|\nabla f_0\|$ is scaled by its much larger initial gradient.

4.3.2 Theoretical and Actual Cost, Storage

We make the cost of L-BFGS concrete with a FLOP count derived from the structure of the algorithm, counting one flop per floating-point addition or multiplication as in [4]. With a full memory of $\bar{m}$ pairs, one iteration costs

$$\text{FLOPs/iter} = \underbrace{(8\bar{m} + 1)\, m}_{\text{two-loop}} + \underbrace{4mn}_{\text{gradient}} + \underbrace{2mn + 4m}_{\text{exact LS}} + \underbrace{2mn}_{\text{objective}} + \underbrace{2m}_{\text{update}},$$

where the objective term is the evaluation of f used by the stagnation check. This is an upper bound: while the memory fills up, and after a reset, the two-loop recursion works on fewer pairs. We therefore also record the number of pairs actually stored at each iteration and compute the flops actually performed. Storage is $(2\bar{m} + 4)\, m$ floats: $2\bar{m}$ vectors of length m for the curvature pairs and four working vectors.

Table 4.11: Total FLOPs to convergence vs. $\bar{m}$: full-memory bound and actual count ($m = 500$, $n = 12$, $\lambda = 0.5$, exact LS, $\varepsilon = 10^{-14}$).

$\bar{m}$	Iter	Mean pairs	Bound	Actual	Storage (MB)
1	20	0.95	1,110,000	1,106,000	0.024
2	17	1.82	1,011,500	999,500	0.032
3	15	2.60	952,500	928,500	0.040
5	14	3.93	1,001,000	941,000	0.056
10	13	5.77	1,189,500	969,500	0.096
12	12	5.50	1,194,000	882,000	0.112
20	12	5.50	1,578,000	882,000	0.176
40	12	5.50	2,538,000	882,000	0.336

The actual cost is minimal and constant from $\bar{m} = 12$ onwards, 0.88M flops. With 12 iterations the memory never holds more than 11 pairs (5.5 on average), so all memories $\bar{m} \geq 12$ perform exactly the same computations, consistently with their identical wall-clock times in Table 4.6. The full-memory bound instead grows linearly with $\bar{m}$ and overestimates the work by up to 2.9× at $\bar{m} = 40$: per-iteration estimates must account for the memory actually in use. Smaller memories cost slightly more in total, from 0.93M flops at $\bar{m} = 3$ to 1.11M at $\bar{m} = 1$: their cheaper iterations do not compensate the few extra iterations they need. Overall the total cost varies by only about 25% over the whole range, and storage is negligible, at most 0.34 MB.

The per-iteration cost is linear in the number of samples m, and, as shown in Section 4.3.3, the iteration count does not grow with m. The direct solver considered here is the dense solution of the normal equations, whose cost grows as $O(m^3)$; the structure-based QR of Chapter 5 reduces the direct cost to $O(nm^2)$, and the two approaches are compared in Chapter 7.

4.3.3 Empirical Scaling with the Dimensions n and m

All previous experiments use the fixed ML-CUP matrix ($m = 500$, $n = 12$). To separate the effect of the dimensions from that of the conditioning, we run two sweeps on synthetic data with a prescribed spectrum: $X = U\Sigma V^T$, where U and V are the orthonormal factors of a Gaussian matrix and the singular values are geometrically spaced from $\sigma_{\max}$ down to $\sigma_{\max}/\rho$. Since $\kappa(\hat{X}) = \sqrt{\sigma_{\max}^2 + \lambda^2}/\lambda$ by Eq. (2.7), fixing $\sigma_{\max}$ and λ makes the condition number *identical* on every row of both sweeps. We set $\sigma_{\max} = 75.74$ and $\lambda = 0.5$, the values of the ML-CUP instance, so that $\kappa(\hat{X}) = 151.5$ throughout, and $\rho = 20$. Prescribing the spectrum means the columns are no longer standardised as in the real data: this is a deliberate trade-off, since controlling κ is what makes the effect of the dimension readable. For the n-sweep we also record the saturation memory $\bar{m}^*$, the smallest memory reaching the minimum iteration count, obtained by testing every $\bar{m}$ from 1 to $n + 10$.

Table 4.12: Scaling with the dimensions, with $\kappa(\hat{X}) = 151.5$ on every row (exact LS, $\varepsilon = 10^{-14}$). The saturation memory $\bar{m}^*$ is recorded only for the n-sweep.

	L-BFGS iter	**CG iter**	$\bar{m}^*$
Varying n, fixed $m = 500$			
$n = 5$	6	7	3
$n = 12$	12	17	11
$n = 25$	25	40	23
$n = 50$	50	93	48
$n = 100$	93	182	92
Varying m, fixed $n = 12$			
$m = 200$	12	17	—
$m = 500$	12	17	—
$m = 2000$	12	17	—
$m = 8000$	12	19	—

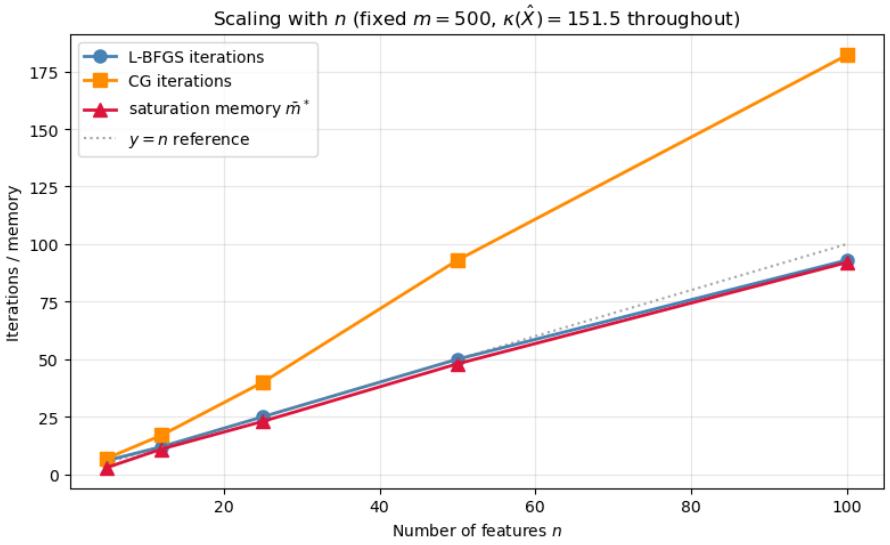


Figure 4.4: Scaling with the number of features n at fixed $m = 500$ and fixed $\kappa(\hat{X}) = 151.5$: L-BFGS iterations, CG iterations and the saturation memory $\bar{m}^*$, against the reference line $y = n$. L-BFGS and $\bar{m}^*$ follow the line closely, while CG departs from it as n grows.

With the conditioning held fixed, the iteration count of L-BFGS is exactly n for $n = 12$, 25 and 50, a direct confirmation of the finite-termination argument of Section 3.6.3: the Hessian restricted to the range of X has n distinct eigenvalues, and the method exhausts them one per iteration. At $n = 100$ the count is 93, slightly below n. This is consistent with the bound, which only gives a maximum: with $\rho = 20$ spread over 100 values, adjacent σ_i^2 become close, and when eigenvalues are tightly clustered the gradient falls below the tolerance before every eigenvalue has been treated individually. At the other end, $n = 5$ needs 6 iterations, one more than the theoretical count: in floating-point arithmetic the last step may leave the gradient just above the tolerance, so that one extra iteration is required. The saturation memory tracks n from just below it (3, 11, 23, 48, 92): the number of curvature pairs needed to reach the minimum iteration count grows with the effective rank, consistently with the real instance of Section 4.2.3, where the saturation point is $\bar{m} = n = 12$. Table 4.13 quantifies what an insufficient memory costs. A single stored pair needs 1.5× the saturated iteration count at $n = 5$ and

2.4× at $n = 100$: the penalty is modest in absolute terms but grows steadily with the dimension, so setting $\bar{m}$ at the effective rank becomes more valuable as n increases.

Table 4.13: Cost of an insufficient memory: iterations with a single stored pair against the saturated count (fixed $m = 500$, exact LS, $\varepsilon = 10^{-14}$).

n	$\bar{m} = 1$	$\bar{m} = \bar{m}^*$	**ratio**
5	9	6	1.5
12	20	12	1.7
25	44	25	1.8
50	107	50	2.1
100	221	93	2.4

Varying m at fixed n and fixed conditioning, the iteration count of L-BFGS is constant at 12 across a forty-fold increase in the sample size, from $m = 200$ to $m = 8000$. Convergence is therefore governed by the effective rank n and is entirely independent of the ambient dimension m; combined with a per-iteration cost linear in m (Section 4.3.2), the total work of L-BFGS grows only linearly with the number of samples.

The comparison with CG is the one result that changes qualitatively with the dimension. On the ML-CUP instance CG needed 15 iterations against the 12 of L-BFGS (Section 4.1.2), a gap of three steps. Here the gap widens monotonically with n: 17 against 12, 40 against 25, 93 against 50, and 182 against 93, so that at $n = 100$ CG performs almost twice as many iterations and exceeds by a factor 1.8 the n steps it would need in exact arithmetic. As in Section 4.1.2, this is not an artefact of the stopping test: checking the true gradient at each iterate rather than the residual updated by recurrence gives exactly the same counts for every n. The explanation is the same erosion of conjugacy by rounding errors, but its effect is far more visible here than on the ML-CUP instance, because the prescribed geometric spectrum spans two orders of magnitude and the Krylov directions become numerically dependent much earlier. L-BFGS, which stores the curvature pairs explicitly instead of relying on the implicit conjugacy of the recurrence, is unaffected and stays close to the theoretical count. The modest advantage observed on the base problem is therefore not a peculiarity of that instance: it is the small-n end of a gap that grows with the effective rank.

4.4 Summary

Our experimental results confirm the theoretical analysis of Section 3.6 and provide several practical insights into the design of L-BFGS for this problem class.

On the *theoretical* side, the most important observation is that the convergence of L-BFGS on our problem is governed by the effective rank of XX^T, not by the condition number. With a memory at least equal to the effective rank, L-BFGS converges in exactly $n = 12$ iterations in every regime, from $\kappa \approx 76$ to $\kappa \approx 7.6 \times 10^5$, to an accuracy of order 10^{-15}, ending with the abrupt collapse that signals finite termination; its direction becomes the Newton direction to machine precision. Steepest descent needs over 1400 iterations and slows down as κ grows, while CG, which would terminate in the same 12 steps in exact arithmetic, needs 15 because rounding errors erode the conjugacy of its directions.

On the *algorithmic* side, exact line search dominates the strong Wolfe variant (12 iterations against 30, and about 4× faster), whose accuracy is limited by its reliance on function values; among the Wolfe constants, only a strict c_2 brings it close to the exact step. The memory improves convergence

monotonically up to the effective rank and then saturates, so $\bar{m} = n$ is the natural choice. With such a memory the H_0 scaling and the restart do not matter; below the effective rank, `bb1` is the most robust scaling and the restart brings no benefit, which motivates the default configuration. These conclusions hold in wall-clock time as well as in iterations.

On the *numerical* side, L-BFGS is insensitive to the starting point and far more accurate than the normal equations at small λ, because its iterates stay in the range of X and it never forms XX^T. Its actual cost is constant for every memory beyond the effective rank, whereas a full-memory FLOP model overestimates it by up to 2.9×, and its storage is negligible. Varying the dimensions with the conditioning held fixed confirms both halves of this picture directly: the iteration count equals the effective rank n and is constant at 12 from $m = 200$ to $m = 8000$, so the total work grows only linearly with the number of samples, while the memory needed tracks n from just below it. The same sweep shows that the advantage of L-BFGS over CG, three iterations on the base problem, widens with n up to almost a factor two at $n = 100$, as rounding errors erode the conjugacy of the CG directions.

Chapter 5

QR Factorization

5.1 QR Solution of the Regularized Least-Squares Problem

We now turn from the optimization point of view to the direct linear algebra point of view. The same regularized least squares problem can be solved without generating an iterative sequence of approximations: one factorizes the augmented matrix

$$A \coloneqq \hat{X} = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} \in \mathbb{R}^{(n+m)\times m}, \qquad b \coloneqq \hat{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \in \mathbb{R}^{n+m}, \tag{5.1}$$

and then solves the resulting triangular problem. Since $\lambda > 0$, Lemma 1 shows that A has full column rank. Therefore the least squares problem

$$\min_{w\in\mathbb{R}^m} \|Aw - b\| \tag{5.2}$$

has a unique minimizer.

5.1.1 Thin QR Factorization

For a full column rank matrix $A \in \mathbb{R}^{p\times m}$, with $p = n + m \geq m$, the thin QR factorization is

$$A = Q_1 R, \qquad Q_1 \in \mathbb{R}^{p\times m}, \quad Q_1^T Q_1 = I_m, \quad R \in \mathbb{R}^{m\times m}, \tag{5.3}$$

where R is upper triangular and nonsingular.
Let $Q_2 \in \mathbb{R}^{p\times n}$ be a matrix whose columns form an orthonormal basis of the orthogonal complement of $\mathrm{col}(Q_1)$. Then

$$Q = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \in \mathbb{R}^{p\times p} \tag{5.4}$$

is an orthogonal matrix:

$$Q^T Q = Q Q^T = I_p. \tag{5.5}$$

Therefore,

$$Q_1^T Q_1 = I_m, \qquad Q_2^T Q_2 = I_n, \qquad Q_2^T Q_1 = 0. \tag{5.6}$$

Hence,

$$Q^T A = Q^T Q_1 R = \begin{bmatrix} Q_1^T Q_1 \\ Q_2^T Q_1 \end{bmatrix} R = \begin{bmatrix} I_m \\ 0 \end{bmatrix} R = \begin{bmatrix} R \\ 0 \end{bmatrix}, \tag{5.7}$$

$$Q^T b = \begin{bmatrix} Q_1^T b \\ Q_2^T b \end{bmatrix} = \begin{bmatrix} c \\ d \end{bmatrix}, \quad c \in \mathbb{R}^m,\ d \in \mathbb{R}^n. \tag{5.8}$$

Orthogonal transformations preserve the Euclidean norm, hence

$$\|Aw - b\|^2 = \left\|Q^T(Aw - b)\right\|^2 = \left\| \begin{bmatrix} Rw - c \\ -d \end{bmatrix} \right\|^2 = \|Rw - c\|^2 + \|d\|^2 . \tag{5.9}$$

Since the term $\|d\|^2$ is independent of w, the minimization problem reduces to

$$\arg\min_{w \in \mathbb{R}^m} \|Aw - b\|^2 = \arg\min_{w \in \mathbb{R}^m} \|Rw - c\|^2 . \tag{5.10}$$

Since R is nonsingular, there exists a unique w such that

$$Rw - c = 0, \qquad \text{that is,} \qquad Rw = c = Q_1^T b. \tag{5.11}$$

Since $A^T A = (Q_1 R)^T (Q_1 R)$, this is the QR analogue of solving the normal equations. Indeed,

$$\begin{aligned} R^T R = A^T A &= \begin{bmatrix} X & \lambda I_m \end{bmatrix} \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} = XX^T + \lambda^2 I_m, \\ R^T c = A^T b &= \begin{bmatrix} X & \lambda I_m \end{bmatrix} \begin{bmatrix} y \\ 0 \end{bmatrix} = Xy. \end{aligned} \tag{5.12}$$

Thus QR and the normal equations define the same solution in exact arithmetic. Numerically, however, QR is preferable because it avoids explicitly forming and solving with $A^T A$, whose condition number is squared:

$$\kappa_2(A^T A) = \kappa_2(A)^2. \tag{5.13}$$

If $X \neq 0$, this distinction is important as $\lambda \to 0^+$, since Eq. (2.7) gives $\kappa_2(A) \sim \sigma_{\max}(X)/\lambda$.
In contrast, QR works directly with the triangular factor R, whose condition number satisfies

$$\kappa_2(R) = \kappa_2(A), \tag{5.14}$$

because the orthogonal factor Q_1 does not affect the singular values. Therefore, QR avoids the loss of accuracy caused by the condition-number squaring introduced by the normal equations.

5.1.2 Algorithmic Structure

The QR-based solver can be summarized as follows:

Algorithm 3 QR Solver for the Regularized Least Squares Problem

Require: Data matrix $X \in \mathbb{R}^{m \times n}$; response $y \in \mathbb{R}^n$; parameter $\lambda > 0$

1: Form the augmented matrix $A = [X^T; \lambda I_m]$ and vector $b = [y; 0]$
2: Compute a thin QR factorization $A \to Q_1 R$
3: Compute $c \leftarrow Q_1^T b$
4: Solve $Rw = c$ by back-substitution
5: **return** w

5.1.3 Cost for the QR Solver

The matrix Q_1 is not formed explicitly. Forming it would require $O((n+m)m)$ storage and applying it as a dense matrix would obscure the sparse and structured nature of A. Instead, the orthogonal transformations used during the factorization are stored in compact form and later applied to b.
For a generic dense matrix $A \in \mathbb{R}^{p\times m}$, with $p = n + m \geq m$, computing a thin QR factorization requires

$$O(pm^2) = O((n+m)m^2) \tag{5.15}$$

floating-point operations.
Once the factorization is available, computing the transformed right-hand side $c = Q_1^T b$ requires at most $O(pm)$ additional operations, whereas solving the upper triangular system $Rw = c$ by back-substitution costs $O(m^2)$. Therefore, the factorization dominates the overall computational cost. Since $p = n + m$, treating the augmented matrix A as a generic dense matrix gives the previously declared $O((n+m)m^2) = O(nm^2 + m^3)$ complexity.
This bound contains a cubic term in m. This is the baseline complexity of the QR solver before exploiting the particular block structure of the augmented matrix. The structured implementation developed in Section 5.2 reduces this cost substantially.

5.2 QR with Householder Reflectors

Householder reflectors are the standard stable mechanism for computing QR factorizations [5, 4]. The implementation stores a normalized reflector vector u, so a Householder reflector has the form

$$H = I - 2uu^T, \qquad \|u\|_2 = 1. \tag{5.16}$$

Equivalently, for an unnormalized nonzero vector v, one may write $H = I - \tau vv^T$ with $\tau = 2/(v^T v)$. In either representation, H is symmetric and orthogonal:

$$H^T = H, \qquad H^T H = I. \tag{5.17}$$

Given a nonzero vector $z \in \mathbb{R}^\ell$, the reflector u is chosen so that all components below the first one are annihilated. To match the implementation and avoid overflow or underflow in the norm calculation, define

$$s = \|z\|_\infty, \qquad \bar{z} = \frac{z}{s}, \qquad \nu = \|\bar{z}\|_2, \qquad \sigma = \operatorname{sign}(z_1) = \begin{cases} 1, & z_1 \geq 0, \\ -1, & z_1 < 0, \end{cases} \tag{5.18}$$

with σ the sign of the first component of the vector, and then

$$\alpha = -\sigma s\nu, \qquad v = \bar{z} + \sigma\nu e_1, \qquad u = \frac{v}{\|v\|_2}. \tag{5.19}$$

The scaling by s makes the norm computation numerically safe, while the factor s must be restored in α because $\|z\|_2 = s\nu$; it does not affect u, since the common scaling factor cancels when the Householder vector is normalized:

$$u = \frac{z - \alpha e_1}{\|z - \alpha e_1\|_2} = \frac{s\,(\bar{z} + \sigma\nu e_1)}{s\,\|\bar{z} + \sigma\nu e_1\|_2} = \frac{v}{\|v\|_2}. \tag{5.20}$$

With H as in Eq. (5.16), this construction gives

$$Hz = \alpha e_1 = \begin{bmatrix} -\operatorname{sign}(z_1)\,\|z\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{5.21}$$

Here $\operatorname{sign}(0) = 1$, as in the definition of σ. The sign choice makes the first component of v an addition of like-signed quantities, thereby avoiding subtractive cancellation. If $z = 0$, the code returns $u = 0$ and $\alpha = 0$ as an identity-reflector sentinel.
At step k, the current active column is therefore mapped to a vector whose entries below position k are zero, while its k-th entry becomes α.

5.2.1 Dense Householder QR

The function `naive_qr_solver` applies Householder QR to the explicit augmented matrix as if it were generic and dense; here *naive* means that the block structure is ignored, not that the transformation is unstable. For $A \in \mathbb{R}^{p\times m}$, the factorization proceeds column by column. At step k, one constructs an active reflector $\widehat{H}_k \in \mathbb{R}^{(p-k+1)\times(p-k+1)}$ such that

$$\widehat{H}_k A_{k:p,k} = \begin{bmatrix} \alpha_k \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{5.22}$$

The same active reflector is applied to the whole trailing submatrix $A_{k:p,k:m}$. Let $H_k = \operatorname{diag}(I_{k-1}, \widehat{H}_k)$ denote its extension to $\mathbb{R}^p$. After m steps,

$$H_m \cdots H_2 H_1 A = \begin{bmatrix} R \\ 0 \end{bmatrix}. \tag{5.23}$$

Since each Householder reflector is orthogonal, their product is also orthogonal. Defining

$$Q^T = H_m \cdots H_2 H_1, \qquad Q \in \mathbb{R}^{p\times p}, \tag{5.24}$$

we obtain

$$Q^T A = \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad R \in \mathbb{R}^{m\times m}, \tag{5.25}$$

where R is upper triangular. Multiplying by Q gives

$$A = Q \begin{bmatrix} R \\ 0 \end{bmatrix}. \tag{5.26}$$

Let $Q_1 \in \mathbb{R}^{p\times m}$ denote the first m columns of Q. Since the remaining rows of the block matrix above are zero, this reduces to the thin QR factorization

$$A = Q_1 R, \qquad Q_1^T Q_1 = I_m. \tag{5.27}$$

The important implementation point is that Q is never formed explicitly. The dense implementation first stores the normalized reflector vectors and then applies them to b, in factorization order:

$$b_{k:p} \leftarrow \widehat{H}_k b_{k:p} = b_{k:p} - 2u_k \left(u_k^T b_{k:p}\right). \tag{5.28}$$

After all m reflectors have been applied, the work vector contains $Q^T b$. Its first m entries are

$$c = Q_1^T b, \tag{5.29}$$

This is the right-hand side of the triangular system $Rw = c$.
The stored reflectors also give products with Q itself. Since $Q^T = H_m \cdots H_1$ and $Q = H_1 \cdots H_m$, a product $Q^T x$ applies the reflectors in factorization order, as above, while a product Qx applies the same reflectors in reverse order, from H_m to H_1 [5, Algorithms 10.2 and 10.3]. Both cost a number of operations proportional to the stored entries, i.e. $O(pm)$, and neither forms Q.

Algorithm 4 Compact Dense Householder QR

Require: Matrix $A \in \mathbb{R}^{p\times m}$ with $p \geq m$
1: **for** $k = 1, \ldots, m$ **do**
2: $z \leftarrow A_{k:p,k}$
3: $(u, \alpha) \leftarrow$ StableHouseholderVector(z) using Eq. (5.18)
4: **if** $u \neq 0$ **then**
5: $A_{k:p,k:m} \leftarrow A_{k:p,k:m} - 2u(u^T A_{k:p,k:m})$
6: **end if**
7: $A_{kk} \leftarrow \alpha$; $A_{k+1:p,k} \leftarrow 0$
8: Store u
9: **end for**
10: $R \leftarrow \text{triu}(A_{1:m,1:m})$
11: **return** R and stored reflectors

Algorithm 4 is the mathematically canonical version. It is useful for explaining QR, but for the project matrix it should be specialized as described next.

5.2.2 Structure-Based Householder QR

The augmented matrix has the same special block structure analyzed before.
The order of the rows is irrelevant for the least-squares problem. Indeed, if P is a permutation matrix, then P is orthogonal and therefore

$$\|Aw - b\| = \|P(Aw - b)\|. \tag{5.30}$$

We can consequently work with the equivalent permuted system

$$\tilde{A} = \begin{bmatrix} \lambda I_m \\ X^T \end{bmatrix}, \qquad \tilde{b} = \begin{bmatrix} 0 \\ y \end{bmatrix}. \tag{5.31}$$

This ordering is particularly convenient because the first m rows already form an upper triangular matrix,

$$R_0 = \lambda I_m. \tag{5.32}$$

The structure-based QR solver exploits the block form $\begin{bmatrix} \lambda I_m \\ X^T \end{bmatrix}$, processing each column with a compact Householder reflector of length $n+1$ without explicitly constructing the full $(m+n) \times m$ augmented matrix. At step k, it forms only the active block

$$B_k = \begin{bmatrix} R_{k,k:m} \\ Z_{:,k:m} \end{bmatrix} \in \mathbb{R}^{(n+1)\times(m-k+1)}. \tag{5.33}$$

A normalized reflector $u_k \in \mathbb{R}^{n+1}$ is computed from the first column of B_k. The update

$$B_k \leftarrow B_k - 2u_k(u_k^T B_k) \tag{5.34}$$

annihilates all n entries of $Z_{:,k}$, the elements in the k-th column of Z. Its first row is written back to $R_{k,k:m}$ and its remaining rows to $Z_{:,k:m}$. Thus the algorithm stores m reflectors of fixed length $n+1$, rather than the decreasing reflectors of length $m+n-k+1$ used by dense QR.

This compact form is exact, not an approximation. At step k, the entries of column k in rows $k+1, \ldots, m$ of the λI_m block are still zero: they are zero in $R_0 = \lambda I_m$, and no previous step has modified them, since reflector $j < k$ acts only on row j of R and on the n rows of Z. The only nonzero entries below the diagonal of column k are therefore the n entries of $Z_{:,k}$, and a reflector supported on row k and on the rows of Z annihilates all of them while leaving rows $k+1, \ldots, m$ of R untouched. The sparsity of the identity block is thus preserved throughout the factorization.

The same reflectors are applied to the permuted right-hand side without forming Q. At step k, only the leading coordinate c_k and the current dense tail $z \in \mathbb{R}^n$ are active:

$$q = \begin{bmatrix} c_k \\ z \end{bmatrix}, \qquad q \leftarrow q - 2u_k(u_k^T q), \qquad c_k \leftarrow q_1, \quad z \leftarrow q_{2:n+1}. \tag{5.35}$$

After the last update, $c \in \mathbb{R}^m$ is the leading part $Q_1^T \tilde{b}$ of $Q^T \tilde{b}$, which is all the triangular solve requires, and the final tail z is the trailing part $d = Q_2^T \tilde{b}$. The implementation returns both, so the full product $Q^T \tilde{b}$ is available; by Eq. (5.9), $\|d\|$ is the minimal residual norm. Products with Q are obtained in the same compact way, applying the reflectors $u_m, \ldots, u_1$ in reverse order to the coordinate pairs $(x_k, x_{m+1:m+n})$. Both products cost $O(nm)$ and never form Q.

Algorithm 5 Structure-Based Householder QR by Columns

Require: $X \in \mathbb{R}^{m\times n}$, $y \in \mathbb{R}^n$, $\lambda > 0$

$R \leftarrow \lambda I_m$; $\quad Z \leftarrow X^T$
for $k = 1, \ldots, m$ **do**
  $B \leftarrow [R_{k,k:m};\, Z_{:,k:m}]$
  $(u_k, \alpha_k) \leftarrow \textsc{StableHouseholderVector}(B_{:,1})$
  if $u_k \neq 0$ **then**
    $B \leftarrow B - 2u_k(u_k^T B)$
  end if
  $B_{11} \leftarrow \alpha_k$; $\quad B_{2:n+1,1} \leftarrow 0$
  $R_{k,k:m} \leftarrow B_{1,:}$; $\quad Z_{:,k:m} \leftarrow B_{2:n+1,:}$
  Store u_k
end for
$c \leftarrow 0 \in \mathbb{R}^m$; $\quad z \leftarrow y$
for $k = 1, \ldots, m$ **do**
  $q \leftarrow [c_k;\, z]$
  if $u_k \neq 0$ **then**
    $q \leftarrow q - 2u_k(u_k^T q)$
  end if
  $c_k \leftarrow q_1$; $\quad z \leftarrow q_{2:n+1}$
end for
$d \leftarrow z$ ▷ Trailing part $Q_2^T\tilde{b}$; $\|d\|$ is the minimal residual
$w \leftarrow \textsc{BackSubstitution}(R, c)$
return $R, w, [c;\, d]$ and the stored reflectors

Chapter 6

QR Factorization Experiments

This chapter evaluates the QR approach of Chapter 5 on the same ML-CUP25 instance used for L-BFGS: $X \in \mathbb{R}^{m \times n}$ with $m = 500$ and $n = 12$, the response vector y drawn with the same fixed seed, and default regularization $\lambda = 0.5$ ($\kappa_2(A) \approx 151$). We compare three solvers for the same least-squares problem: the dense Householder QR of the augmented matrix, which ignores its block structure; the structure-based Householder QR of Section 5.2.2, which is the solver proposed for this project; and the normal equations $(XX^T + \lambda^2 I_m)\, w = Xy$ solved with `numpy.linalg.solve`, used as an external baseline.

Before any timing or stability study, we verify that the implementation is correct on the default instance. A single Householder reflector annihilates the subdiagonal part of its column and is orthogonal to working precision (errors of order 10^{-16}). Measured against an independent SVD reference, both QR solvers reach a relative error of about 4×10^{-15}, while the normal-equations solution is already at 5×10^{-13}, a first sign of its squared condition number. Finally, the implicit products with Q and Q^T are exact to working precision: for both solvers $\left\|Q(Q^T v) - v\right\| / \|v\|$ and the relative error of the reconstruction $Q\,[R; 0] = A$ are below 10^{-15}, and for the structure-based solver the norm of the trailing part of $Q^T \tilde{b}$ matches the minimal residual to 6×10^{-17}, without ever forming Q.

Each reported time is the median of ten runs after a discarded warm-up run, and data generation and accuracy diagnostics are always excluded from the timed regions.

The chapter is organised in two parts. We first study the *computational cost* of the structure-based solver: a detailed operation count, its asymptotic comparison with dense QR and the normal equations, and the measured runtimes for increasing problem size m (Section 6.1). We then study its *numerical stability* as the conditioning of the problem is varied through λ, comparing forward errors and optimality residuals against an independent SVD reference (Section 6.2).

6.1 Runtime Scalability

6.1.1 Detailed Operation Count

We now give a precise floating-point operation (FLOP) count for the structure-based solver, beyond the asymptotic order, counting one flop per floating-point addition or multiplication as for L-BFGS (Section 4.3.2). At step k the active block is $(n + 1) \times (m - k + 1)$, and the work splits into three phases.

Reflector construction: Computing the compact Householder vector u_k from the first column (length $n+1$) requires scaling, one Euclidean norm, a sign choice, and a normalisation:

$$\text{build}(u_k) \;\approx\; 6(n+1) \text{ FLOP},$$

independent of the column index k. Over all m columns this contributes $6(n+1)\,m$, linear in m.

Block update: Applying $H_k = I - 2u_k u_k^T$ to the active block of width $w_k = m - k + 1$ is the dominant cost. The product $u_k^T B_k$ costs $2(n+1)w_k$, the outer product and the rank-one subtraction cost a further $2(n+1)w_k$, for

$$\text{update at step } k \;\approx\; 4(n+1)(m-k+1) \text{ FLOP}.$$

Summing over $k = 1, \ldots, m$,

$$\sum_{k=1}^{m} 4(n+1)(m-k+1) \;=\; 4(n+1)\frac{m(m+1)}{2} \;\approx\; 2(n+1)\,m^2 \text{ FLOP},$$

which is the leading term of the factorisation.

Right-hand side and solve: Applying the m stored reflectors to $\tilde{b}$ touches only an $(n+1)$-vector at each step, costing $\approx 7(n+1)\,m$ in total; the final back-substitution on the $m \times m$ triangular factor costs m^2. Both are dominated by the factorisation for $m \gg n$.

Total: Collecting the three phases,

$$\text{FLOP}_{\text{struct}}(m,n) \;\approx\; \underbrace{2(n+1)m^2}_{\text{factorisation}} + \underbrace{m^2}_{\text{back-subst.}} + \underbrace{13(n+1)m}_{\text{reflectors + rhs}} \;=\; \big(2(n+1)+1\big)m^2 + O(nm). \tag{6.1}$$

With $n = 12$ the leading constant is $2(n+1)+1 = 27$, so the solver performs about $27\,m^2$ FLOP. Table 6.1 evaluates Eq. (6.1) at the sampled sizes.

Table 6.1: Theoretical FLOP counts ($n = 12$): structure-based QR from Eq. (6.1); dense Householder QR of the $(m+n) \times m$ augmented matrix, $\approx 2(m+n)m^2 - \frac{2}{3}m^3$; normal equations solved by LU as in `numpy.linalg.solve`, $\approx 2nm^2 + \frac{2}{3}m^3$. The last column is the ratio normal equations / structure-based.

m	**Structure-based**	**Dense QR**	**Normal eq.**	**Ratio**
100	2.9×10^5	1.6×10^6	9.3×10^5	3.2×
500	6.8×10^6	1.7×10^8	9.0×10^7	13.1×
1000	2.7×10^7	1.4×10^9	6.9×10^8	25.5×
2000	1.1×10^8	1.1×10^{10}	5.4×10^9	50.2×
4000	4.3×10^8	8.6×10^{10}	4.3×10^{10}	99.6×

The count makes the structural advantage quantitative rather than asymptotic. Both the structure-based solver and the normal equations share the $O(nm^2)$ term from touching the dense block, but the normal equations add a cubic $\frac{2}{3}m^3$ term from the dense LU factorisation, and dense QR an even larger one, both absent here because the λI_m block is already triangular. The resulting advantage grows with m: from about 3× at $m = 100$ to about 100× at $m = 4000$ over the normal equations, matching the $O(nm^2)$ versus $O(m^3)$ gap of Eq. (6.3).

6.1.2 Asymptotic Cost and Timings

The column-oriented structure-based method applies one reflector to an $(n+1) \times (m-k+1)$ block at each step. Its factorization cost is therefore $\sum_{k=1}^{m} O\big((n+1)(m-k+1)\big) = O(nm^2 + m^2)$.
Applying the reflectors to the right-hand side costs $O(nm)$ and back substitution costs $O(m^2)$. The complete structure-based solver thus costs

$$O(nm^2 + m^2). \tag{6.2}$$

Combining these bounds with Section 5.1.3 gives

$$\begin{aligned} T_{\text{dense QR}}(m,n) &= O(m^3 + nm^2), \\ T_{\text{structure-based QR}}(m,n) &= O(m^2 + nm^2), \\ T_{\text{normal equations}}(m,n) &= O(m^3 + nm^2). \end{aligned} \tag{6.3}$$

Hence, with the project value $n = 12$ fixed, the structure-based algorithm is quadratic in m, whereas generic dense QR and a dense normal-equations solve retain a cubic term.

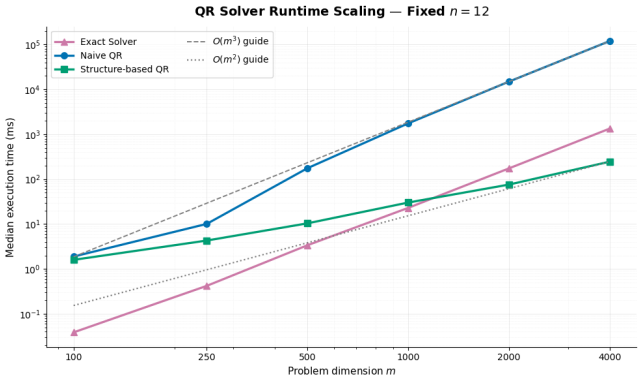


Figure 6.1: Median solver times for increasing m, with $n = 12$ fixed.

The structure-based implementation stores the triangular factor, a copy of the dense block, and m reflectors of length $n+1$, for $O(m^2 + nm)$ memory.
The notebook first constructs a single matrix with 4000 rows. The original 500 rows are retained as the leading block; each additional row is sampled as a complete row of X and perturbed by Gaussian noise scaled by the empirical standard deviation of each column. Taking prefixes of this matrix produces nested problems with fixed $n = 12$, $m \in \{100, 250, 500, 1000, 2000, 4000\}$, and $\lambda = 0.5$. Data generation is performed once, outside every timed region. Each entry in Table 6.2 is the median of ten runs after a discarded warm-up run.

Table 6.2: Internal solver times in milliseconds for the augmented-data runtime sweep. The normal-equations timer is partial.

m	**Normal eq.**	**Dense QR**	**Structure-based QR**
100	0.039	1.891	1.596
250	0.421	10.094	4.296
500	3.379	175.910	10.392
1000	22.975	1754.163	30.200
2000	174.064	14927.239	75.879
4000	1340.009	119515.903	245.618

The QR timings include the preparation of the required data structures, factorization, transformation of the right-hand side, and back substitution. Dataset generation and accuracy diagnostics are excluded from the timed regions. The normal-equations timer, instead, covers only `numpy.linalg.solve` and excludes the formation of $XX^T + \lambda^2 I_m$ and Xy, which favours this baseline.

Figure 6.1 highlights the growing runtime gap between dense QR and the structure-based method, which is the faster of the two QR solvers at every sampled size, reaching a speedup of about 487× over dense QR at $m = 4000$. The cubic versus quadratic separation is clear on the final doubling from $m = 2000$ to $m = 4000$: dense QR and the normal equations grow by factors of about 8.0 and 7.7, close to the $2^3 = 8$ expected of an $O(m^3)$ method, against the 3.2 of structure-based QR, below the 2^3 of a cubic method and approaching the $2^2 = 4$ of an $O(m^2)$ one, consistent with Eq. (6.3).

The detailed operation count of Section 6.1.1 predicts a leading term of $\big(2(n+1)+1\big)m^2 \approx 27\,m^2$ floating-point operations for $n = 12$, so the work quadruples on each doubling of m. The measured times grow more slowly, by factors between 2.5 and 3.2 from $m = 500$ to $m = 4000$: the time per predicted flop decreases from about 1.5 to 0.6 ns. The solver executes a Python loop over the m columns, each step paying a fixed interpretation overhead and acting on a block of only $n+1 = 13$ rows; as m grows, the blocks become wider, the overhead is amortised and the arithmetic dominates. The quadratic growth predicted by the model is therefore approached from below, and becomes apparent at the largest sizes.

The normal equations are faster than the structure-based solver up to $m = 1000$, and slower from $m = 2000$ onwards, by 2.3× at $m = 2000$ and 5.5× at $m = 4000$, a gap that grows with m as the flop count predicts. At small m their advantage has two sources: their timer is partial, as noted above, and `numpy.linalg.solve` runs a compiled, blocked LAPACK factorization that executes far more operations per second than our Python loop. As m grows, however, their cubic term outweighs this efficiency: flop counts do not predict the absolute ratio between implementations of different nature, but they do predict how it evolves.

6.2 Algorithmic Stability and the Condition Number

Conditioning is a property of the mathematical problem, whereas stability is a property of the algorithm used to solve it. For $A = [X^T; \lambda I_m]$,

$$A^T A = XX^T + \lambda^2 I_m. \tag{6.4}$$

Given $\lambda > 0$, the condition number of A is

$$\kappa_2(A) = \|A\|_2 \|A^\dagger\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} = \sqrt{\frac{\lambda_{\max}(XX^T + \lambda^2 I_m)}{\lambda_{\min}(XX^T + \lambda^2 I_m)}} = \frac{\sqrt{\sigma_{\max}(X)^2 + \lambda^2}}{\lambda}. \tag{6.5}$$

The singular values of A are the square roots of the eigenvalues of $A^T A = XX^T + \lambda^2 I_m$. Since $m > n$, XX^T has a zero eigenvalue, hence $\lambda_{\min}(A^T A) = \lambda^2$, the eigenvalue shift introduced by regularization. If $X \neq 0$, Eq. (6.5) behaves as $\sigma_{\max}(X)/\lambda$ when $\lambda \to 0^+$ and tends to 1 as $\lambda \to \infty$. Forming the normal equations squares this number, as shown in Eq. (5.13); this is precisely the conditioning penalty avoided by Householder QR, getting $\kappa_2(A)$ instead of $\kappa_2(A)^2$.

In the standard floating-point model with machine precision $\varepsilon_{\text{mach}}$, Householder QR is backward stable: up to dimension-dependent constants, the computed factors are the exact QR factors of $A + \Delta A$ with

$$\frac{\|\Delta A\|_2}{\|A\|_2} = O(\varepsilon_{\text{mach}}), \tag{6.6}$$

and the computed orthogonal factor is orthogonal to working precision. The resulting forward error still depends on the conditioning and geometry of the least-squares problem. Accordingly, the notebook uses $\varepsilon_{\text{mach}}\kappa_2(A)$ for Householder QR and $\min\{1, \varepsilon_{\text{mach}}\kappa_2(A)^2\}$ for the normal equations as heuristic indicators of the expected forward-error scale. These quantities should not be interpreted as rigorous universal upper bounds. In particular, the forward sensitivity of a least-squares solution depends not only on $\kappa_2(A)$, but also on the geometry of the problem, including the size and orientation of the residual. The curves are therefore intended only to illustrate the expected conditioning trend of the two methods. The notebook additionally plots $\varepsilon_{\text{mach}}$ as a reference scale for η_{opt}.
The notebook diagnoses this behavior using an independent thin-SVD reference. If $X = U\Sigma V^T$, then the Tikhonov solution is

$$w_{\text{SVD}} = U \operatorname{diag}\left(\frac{\sigma_i}{\sigma_i^2 + \lambda^2}\right) V^T y. \tag{6.7}$$

For a computed solution w, the two reported diagnostics are the relative forward error

$$e_{\text{fwd}} = \frac{\|w - w_{\text{SVD}}\|_2}{\|w_{\text{SVD}}\|_2} \tag{6.8}$$

and, for the augmented residual $r = Aw - b \neq 0$, the scaled first-order optimality residual

$$\eta_{\text{opt}} = \frac{\left\|A^T r\right\|_2}{\|A\|_2 \, \|r\|_2}. \tag{6.9}$$

At an exact least-squares solution $A^T r = 0$, so a small η_{opt} verifies the optimality condition. It does not by itself guarantee a small forward error when A is severely ill-conditioned; both diagnostics must therefore be considered together.

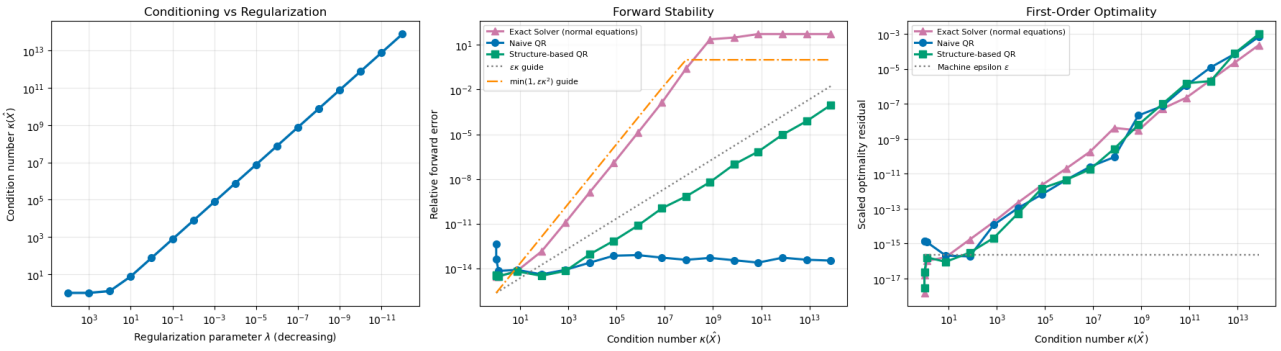


Figure 6.2: Conditioning and numerical stability: condition number versus λ (left), relative forward error (middle), and scaled optimality residual (right).

The experiment in Figure 6.2 sweeps $\lambda \in \{10^j : j = -12, \ldots, 4\}$. The left panel shows the growth of the condition number as λ decreases, from $\kappa_2(A) \approx 1$ at $\lambda = 10^4$ to $\kappa_2(A) \approx 7.6 \times 10^{13}$ at $\lambda = 10^{-12}$. The middle panel shows that the normal equations lose accuracy much earlier than both Householder methods. At $\lambda = 10^{-6}$ ($\kappa_2(A) \approx 7.6 \times 10^7$) the normal equations already have a relative forward error of 2.5×10^{-1}, so that not even the leading digit is reliable, whereas structure-based QR attains 6.7×10^{-10}. At $\lambda = 10^{-12}$ the normal-equations error saturates around 54, while structure-based QR still retains about three correct digits (9.0×10^{-4}).

To make this conditioning penalty quantitative, we fit the forward error against $\kappa_2(A)$ in log–log scale over the intermediate range $\kappa_2(A) \in [10^4, 10^8]$, large enough for the error to grow but below the saturation of the normal equations. The fitted slopes are 2.10 for the normal equations and 1.01 for structure-based QR, recovering the theoretical exponents: forming A^TA makes the error grow as $\kappa_2(A)^2$ (Eq. (5.13)), whereas QR works directly with the triangular factor and degrades only as $\kappa_2(A)$. Dense QR behaves differently: its forward error remains at the 10^{-14} level throughout the ill-conditioned regime (fitted slope ≈ 0), and never exceeds 5×10^{-13} over the whole sweep. Retaining all $m + n$ rows makes it the most accurate method in the ill-conditioned regime, but its $O(m^3)$ cost (Section 6.1) makes it impractical for large m. This near-machine-precision behaviour is an empirical property of this problem and should not be read as a conditioning-independent guarantee. The structure-based solver therefore trades a modest amount of accuracy, growing as $\varepsilon_{\text{mach}}\kappa_2(A)$, for a cost that is quadratic rather than cubic in m, while remaining dramatically more accurate than the normal equations.

The right panel shows why the two diagnostics must be read together. At $\lambda = 10^{-12}$ the normal equations attain the *smallest* scaled optimality residual of the three solvers (2.3×10^{-4}), yet their forward error at the same point is the *largest* by many orders of magnitude. A small η_{opt} certifies that $A^Tr \approx 0$ for the computed solution, but on a severely ill-conditioned problem this does not imply closeness to the true minimiser: the normal equations accurately solve a perturbed problem while landing far from w_{SVD}. The forward error is therefore the decisive diagnostic, and by that measure both Householder methods dominate the normal equations across the ill-conditioned range.

6.3 Summary

Our experiments confirm that exploiting the block structure of the augmented matrix makes Householder QR both cheaper and, compared with the normal equations, far more robust, as predicted in Chapter 5.

On the side of *computational cost*, the detailed operation count shows that the structure-based solver performs about $27\,m^2$ flops for $n = 12$, with no cubic term, because the λI_m block is already triangular and every reflector has fixed length $n + 1$. Dense QR and the normal equations instead retain a cubic term, so the theoretical advantage grows linearly with m, from about $3\times$ to about $100\times$ over the normal equations in the sampled range. The measured runtimes follow these predictions: on the final doubling from $m = 2000$ to $m = 4000$, dense QR and the normal equations grow by about 8.0 and 7.7, and the structure-based solver by about 3.2. At $m = 4000$ the structure-based solver is about $487\times$ faster than dense QR. Its growth approaches the quadratic rate from below, because the fixed cost of its Python loop over the columns is amortised as m grows. The normal equations, which run in compiled LAPACK code and are timed only partially, are faster up to $m = 1000$; from $m = 2000$ onwards the cubic term dominates, and the structure-based solver becomes faster, by $5.5\times$ at $m = 4000$.

On the side of *numerical stability*, the sweep over λ shows the effect of the condition-number squaring of the normal equations. At $\lambda = 10^{-6}$ ($\kappa_2(A) \approx 7.6 \times 10^7$) the normal equations no longer have a reliable leading digit, whereas the structure-based solver attains a relative forward error of 6.7×10^{-10}; at $\lambda = 10^{-12}$ it still retains about three correct digits, while the normal-equations error has saturated. A log–log fit recovers the theoretical exponents, with slopes 2.10 for the normal equations and 1.01 for structure-based QR. Dense QR is the most accurate method, remaining at the 10^{-14} level, but at an $O(m^3)$ cost. The structure-based solver therefore offers the best trade-off: quadratic cost in m and an

error that grows only as $\varepsilon_{\text{mach}}\kappa_2(A)$. Finally, a small scaled optimality residual alone does not certify an accurate solution on ill-conditioned problems, so both diagnostics must be read together.

Chapter 7

Comparison: QR vs. L-BFGS

This chapter compares L-BFGS and structure-based Householder QR under a common experimental protocol. Both solve the same regularized least-squares problem, but their computational costs and sensitivity to the spectrum differ. We therefore compare complete solve times together with coefficient errors, optimality residuals, and termination status. All results and both figures below come from the executed notebook `04_comparison.ipynb`.

7.1 Experimental Protocol and Accuracy Measures

The initial problem is the standardized ML-CUP25 matrix with $m = 500$, $n = 12$, and $\lambda = 0.5$, using the same synthetic response vector as the preceding experiments (seed 42). Consequently, accuracy here concerns the numerical solution of the optimization problem, rather than prediction on unseen data. L-BFGS uses the configuration selected in Chapter 4: memory $\bar{m} = 12$, BB1 scaling, exact line search, no curvature-based restart, and $w_0 = 0$. Unless explicitly varied, the relative gradient tolerance is $\varepsilon = 10^{-14}$ and the iteration budget is 2000. The curvature-pair acceptance safeguard remains active.

Each complete solver call is timed externally after one discarded warm-up, with ten repetitions and a reproducibly shuffled method order. The reported statistic is the median; the interval between the first and third quartiles describes timing dispersion, not a confidence interval. Initialization, factorization, and back substitution are included where applicable. Data construction, SVD references, and accuracy diagnostics are excluded. The run uses double precision, Python 3.13.9, NumPy 2.1.3, and one BLAS thread on an ARM64 machine. These timings describe the present implementations and should not be directly substituted for the internal timings in the individual algorithm chapters.

The independent thin-SVD solution of Eq. (6.7) supplies a common numerical reference w_{SVD}. We use the forward error of Eq. (6.8) and the relative gradient

$$e_w = \frac{\|w - w_{\mathrm{SVD}}\|_2}{\|w_{\mathrm{SVD}}\|_2}, \qquad g_{\mathrm{rel}} = \frac{\left\|X(X^T w - y) + \lambda^2 w\right\|_2}{\|Xy\|_2}. \tag{7.1}$$

Since $w_0 = 0$, the latter agrees with the stopping normalization in Eq. (3.19). It differs from η_{opt} in Eq. (6.9), whose denominator contains the augmented residual. A QR result is labelled as a direct solve; it is assessed with the same metrics, without assigning it an iterative stopping tolerance.

The notebook also records fitted-vector discrepancies, data and augmented residuals, and a scaled stationarity residual. Agreement in the objective alone is insufficient near convergence. In particular,

writing $e = w - w_{\mathrm{SVD}}$ and $H = XX^T + \lambda^2 I_m$ gives

$$f(w) - f(w_{\mathrm{SVD}}) = \nabla f(w_{\mathrm{SVD}})^T e + \tfrac{1}{2} e^T H e. \tag{7.2}$$

The nonnegative quantity $e^T He/(2f(w_{\mathrm{SVD}}))$ avoids subtracting nearly equal objectives, but equals the relative objective gap only when the reference is an exact minimizer. The SVD reference itself has $g_{\mathrm{rel}} \approx 2.0 \times 10^{-15}$ on the initial instance.

7.2 Accuracy and Runtime on ML-CUP25

Table 7.1 shows that both algorithms reach coefficient errors of order 10^{-15}. L-BFGS satisfies the requested tolerance in 12 iterations and is approximately 21.7 times faster in this run. Differences between forward errors at this scale do not establish a meaningful accuracy advantage over a floating-point SVD reference.

Table 7.1: Complete solve times and achieved accuracy on ML-CUP25 ($m = 500$, $n = 12$, $\lambda = 0.5$). Times and quartiles are in milliseconds.

Method	**Median**	$[Q_{0.25}, Q_{0.75}]$	e_w	g_{rel}
L-BFGS	0.512	$[0.499, 0.522]$	2.03×10^{-15}	4.26×10^{-17}
Structure-based QR	11.114	$[11.035, 11.340]$	2.64×10^{-15}	6.49×10^{-16}

Both solutions give $f(w) \approx 0.092632$, a relative data residual of 0.029902, and a relative augmented residual of 0.161038. These nonzero residuals are compatible with optimality of the regularized problem. In particular, minimizing the data residual alone would ignore the penalty on w.

7.2.1 Time at a Common Accuracy Requirement

Figure 7.1 varies only the L-BFGS tolerance, from 10^{-2} to 10^{-14}. The three tightest tolerances all produce the same 12-iteration solution; their small timing differences are measurement variability. QR contributes one complete direct solve, rather than an iterative convergence trajectory.



Figure 7.1: Measured time–accuracy trade-off on the initial problem. Labels indicate requested L-BFGS tolerances. All sampled L-BFGS runs satisfy their tolerance; the square denotes the direct QR result.

To compare the cost of reaching a common quality level, impose $e_w \leq 10^{-6}$ and $g_{\text{rel}} \leq 10^{-8}$. Among the sampled configurations, the fastest qualifying L-BFGS run requests $\varepsilon = 10^{-8}$ and takes 10 iterations: its median time is 0.388 ms, with $e_w = 1.41 \times 10^{-7}$ and $g_{\text{rel}} = 8.69 \times 10^{-9}$. QR also meets both targets, taking 11.114 ms, so the measured time ratio is approximately 28.7. This is a comparison at common acceptance thresholds, rather than identical achieved errors or a claim of globally optimal parameter tuning.

7.3 Dimensional Scaling and Storage

7.3.1 Increasing the Number of Unknowns

The m sweep follows the nested row-resampling construction of Section 6.1: original rows are retained, additional rows are sampled with a small perturbation, and prefixes give $m \in \{100, 250, 500, 1000, 2000, 4000\}$ at fixed $n = 12$ and $\lambda = 0.5$. Unlike the prescribed-spectrum study in Section 4.3.3, conditioning can change with m.



Figure 7.2: Complete solve times, coefficient errors, and L-BFGS iteration counts for increasing m. Dashed complexity guides are normalized at the smallest size and are not fitted asymptotic laws.

L-BFGS takes 12 iterations at every sampled size and meets its tolerance throughout. Its time rises from 0.431 ms at $m = 100$ to 1.745 ms at $m = 4000$, whereas QR rises from 1.801 to 237.894 ms. The corresponding QR/L-BFGS ratio grows from 4.2 to 136.3. Both methods retain forward errors below 10^{-14} over this sweep.

The growing runtime separation is consistent with the cost models of Sections 4.3.2 and 6.1.1:

$$T_{\text{L-BFGS}} = O(k(mn + \bar{m}m)), \qquad T_{\text{QR}} = O\big((n+1)m^2\big). \tag{7.3}$$

The first bound depends on the observed iteration count k; linear growth in m requires that this count remain bounded. Fixed overheads and the finite range of sizes prevent interpreting the measured curves as exact power laws.

Storage also favors L-BFGS in this regime. Counting only the principal double-precision arrays gives $8(2\bar{m}m)$ bytes for its pair capacity and $8(m^2 + m(n+1))$ bytes for QR's dense triangular-factor array and compact reflectors. At $m = 4000$ these are 0.732 and 122.467 MiB, respectively. Inputs, temporary arrays, Python objects, and convergence histories are excluded; these figures are array counts, not measured peak process memory.

7.3.2 Increasing the Number of Spectral Directions

A separate synthetic sweep fixes $m = 500$, $\lambda = 0.5$, and $\bar{m} = 12$, while increasing n from 4 to 96. Singular values are geometrically spaced between 1 and 10, keeping $\kappa_2(A) \approx 20.0$ and $\kappa_2(H|_{\text{range}(X)}) = 80.2$ constant. This differs from the spectrum and memory study in Section 4.3.3.

The L-BFGS iteration counts are $4, 8, 12, 33, 78, 113$ for $n = 4, 8, 12, 24, 48, 96$, respectively. All runs satisfy the requested tolerance, but fixed memory becomes less effective as more spectral directions are introduced. At $n = 96$, L-BFGS takes 10.255 ms and QR 27.517 ms, reducing the timing advantage to approximately 2.7. Their forward errors are 1.91×10^{-14} and 3.62×10^{-15}, respectively. Thus the favorable behavior at $n = 12$ should not be extrapolated with a constant iteration count to larger n.

7.4 Conditioning and a Difficult Synthetic Spectrum

On ML-CUP25, decreasing λ from 0.5 to 10^{-8} raises the full augmented condition number to approximately 7.6×10^9. Nevertheless, L-BFGS still converges in 12 iterations with $e_w \approx 2.1 \times 10^{-15}$. Structure-based QR instead gives $e_w \approx 7.7 \times 10^{-8}$, despite $g_{\text{rel}} \approx 5.6 \times 10^{-16}$. This illustrates why a tiny optimality residual does not guarantee accurate coefficients.

The subspace structure explains why the full condition number alone does not predict L-BFGS behavior here. Starting from zero, in exact arithmetic its iterates remain in range(X), where

$$\kappa_2\big(H|_{\text{range}(X)}\big) = \frac{\sigma_{\max}(X)^2 + \lambda^2}{\sigma_{\min}(X)^2 + \lambda^2} \tag{7.4}$$

for full-column-rank X. This ratio remains about 10^3 as $\lambda \to 0$ for the present data, while the full Hessian has additional eigenvalues λ^2. The observed 12-step convergence is not a general finite-termination guarantee for L-BFGS: the changing initial scaling, limited memory, curvature safeguards, and rounding distinguish it from the exact-arithmetic result discussed in Section 3.6.3.

To test a less favorable restricted spectrum, we construct one matrix with $m = 160$, $n = 24$, and singular values geometrically spaced from 10^{-4} to 1. Table 7.2 reports the results for three regularization values, keeping the L-BFGS settings unchanged.

Table 7.2: Synthetic spectral stress test. Times are medians in milliseconds; the asterisk marks exhaustion of the 2000-iteration budget. QR has no iterative stopping criterion.

λ	**Method**	**Time**	k	e_w	g_{rel}
10^{-4}	L-BFGS	111.577	2000*	3.18×10^{-5}	9.23×10^{-8}
	Structure-based QR	3.354	–	3.95×10^{-13}	8.96×10^{-14}
10^{-2}	L-BFGS	2.997	57	3.92×10^{-13}	6.91×10^{-15}
	Structure-based QR	3.272	–	1.70×10^{-14}	1.17×10^{-15}
1	L-BFGS	0.319	9	4.62×10^{-15}	5.17×10^{-16}
	Structure-based QR	3.273	–	4.56×10^{-15}	5.72×10^{-16}

At $\lambda = 10^{-4}$, the restricted Hessian condition number is approximately 5×10^7. L-BFGS reaches its budget without satisfying the tolerance, while QR is about 33 times faster and gives a forward error almost eight orders of magnitude smaller. QR's relative gradient is still above 10^{-14}, so its advantage should be stated in terms of the achieved metrics, rather than attributed to satisfying the L-BFGS stopping test. At $\lambda = 10^{-2}$, L-BFGS converges in 57 iterations and the times are similar, with QR more accurate in the coefficients. At $\lambda = 1$, both reach errors of order 10^{-15} and L-BFGS is faster again. The weakest regularization in this synthetic test is the only L-BFGS configuration in the recorded experiments that misses its requested tolerance.

7.5 Conclusions

- **For the tested ML-CUP25 problems**, L-BFGS combines coefficient accuracy near working precision with shorter runtimes and substantially smaller principal storage requirements.
- **As the spectral dimension increases**, fixed L-BFGS memory requires more iterations, reducing its runtime advantage even at controlled conditioning.
- **For the difficult weakly regularized synthetic problem**, structure-based QR is faster and more accurate than the budget-limited L-BFGS run. On the real-data small-λ sweep, the accuracy ranking reverses.

The preferred solver therefore depends on the spectrum, regularization, required accuracy, and available storage. These experiments support a conditional choice between the two implementations, rather than a uniform ranking of direct and iterative methods.

Appendix A

Proofs

Lemma 1 ($\hat{X}$ has full column rank)**.** *If $\lambda > 0$, then $\hat{X}^T\hat{X} = XX^T + \lambda^2 I_m \succ 0$.*

Proof. For any nonzero $v \in \mathbb{R}^m$: $v^T(XX^T + \lambda^2 I_m)v = \left\|X^T v\right\|^2 + \lambda^2 \|v\|^2 \geq \lambda^2 \|v\|^2 > 0$. □

Lemma 2 (Eigenvalue shift)**.** *If α is an eigenvalue of A, then $\alpha + c$ is an eigenvalue of $A + cI$.*

Proof. $Av = \alpha v \iff (A + cI)v = (\alpha + c)v$. □

Lemma 3 (Singular values of XX^T)**.** *The eigenvalues of $XX^T \in \mathbb{R}^{m\times m}$ are $\{\sigma_1^2, \ldots, \sigma_n^2, 0, \ldots, 0\}$, where $\sigma_1, \ldots, \sigma_n$ are the singular values of X. In particular, $\lambda_m(XX^T) = 0$ when $m > n$.*

Proof. Let $X = U\Sigma V^T$ be the SVD. Then $XX^T = U\Sigma\Sigma^T U^T$, where $\Sigma\Sigma^T = \operatorname{diag}(\sigma_1^2, \ldots, \sigma_n^2, 0, \ldots, 0) \in \mathbb{R}^{m\times m}$. □

Bibliography

[1] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 1st ed. New York: Springer, 1999.

[2] Jonathan Barzilai and Jonathan M. Borwein. "Two-point step size gradient methods". In: *IMA Journal of Numerical Analysis* 8.1 (1988), pp. 141–148.

[3] Dong C. Liu and Jorge Nocedal. "On the limited memory BFGS method for large scale optimization". In: *Mathematical Programming* 45 (1989), pp. 503–528.

[4] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. 4th ed. Johns Hopkins University Press, 2013.

[5] Lloyd N. Trefethen and David Bau III. *Numerical Linear Algebra*. SIAM, 1997.